

INTRODUCCIÓN A IOT

MANUAL DE PRÁCTICAS:
Uso de protocolos MQTT
y HTTP para conexión
con diversas aplicaciones.

Manual para las asignaturas:

- ACP-137-Tema Selecto: Introducción a IoT
- AFDE-PPR-23-Tema Selecto: Principios de IoT

Manual de prácticas para la Unidad
IV-Aplicaciones de IoT

AUTORES:

M.M. Jesús Orifiel Alvarez Ruiz
Dr. Víctor Manuel Sánchez Huerta
Mtro. Set Jubal Castillo Ávila



UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE QUINTANA ROO

ÍNDICE

PRÁCTICA I – ESP32 CON MQTT	6
PRÁCTICA II – ESP32 Y CONEXIÓN A UNA BASE DE DATOS	25
PRÁCTICA III – TELEGRAM Y ESP32	49
PRÁCTICA IV - CONEXIÓN A WHATSAPP	61

ÍNDICE DE FIGURAS

Ilustración 1.1 – Funcionamiento del bróker. (Fuente: What is MQTT, and Why Does it Matter? Kymera Systems Inc)	7
Ilustración 1.2 – Diagrama a implementar.....	8
Ilustración 1.3 – Pagina del MQTTX.....	9
Ilustración 1.4 – Datos Broker y TCP Port.	9
Ilustración 1.5 – Hacer click en “Use Online”	15
Ilustración 1.6 – Creamos la conexión	15
Ilustración 1.7 – Configurando la conexión del cliente	16
Ilustración 1.8 – Estado de conexión exitosa.	17
Ilustración 1.9 – Suscribiéndose a un tópico	18
Ilustración 1.10 – Visualizando lo que envía la ESP32 a los tópicos.	19
Ilustración 1.11 – Espacio donde publicamos en los tópicos	19
Ilustración 1.12 – A la espera de enviar el comando	19
Ilustración 1.13 – Comando enviado y nuevo estado del LED.....	20
Ilustración 1.14 – App que permite la visualización y control de los tópicos de manera más ordenada y sencilla.	21
Ilustración 2.1 – Arduino IoT Cloud	26
Ilustración 2.2 – Diagrama a implementar.....	27
Ilustración 2.3 – Pestaña de creación de un proyecto	28
Ilustración 2.4 – Pestaña para desactivar la asistencia de IA	28
Ilustración 2.5 – Pestaña para habilitar Google Analytics.....	29
Ilustración 2.6 – Cargando la configuración del proyecto	29
Ilustración 2.7 – Pantalla de inicio de la página de consola	30
Ilustración 2.8 – Pestaña para autenticación	31
Ilustración 2.9 – Métodos para autenticarte en la aplicación	31
Ilustración 2.10 – Vincular la cuenta y guardar la configuración	32
Ilustración 2.11 – Apartado para agregar un usuario	32
Ilustración 2.12 – Apartado para registrar un usuario	33

Ilustración 2.13 – Lista de usuarios	33
Ilustración 2.14 – Pestaña para crear la base de datos.....	34
Ilustración 2.15 – Seleccionar la ubicación de la base datos	34
Ilustración 2.16 – configuración de las reglas de seguridad.....	35
Ilustración 2.17 – Copiar la URL de la base de datos	35
Ilustración 2.18 – Apartado de la configuración del proyecto	36
Ilustración 2.19 – Copiar la clave de la API	36
Ilustración 2.20– Configuración de la biblioteca	37
Ilustración 2.21 – Cargar el proyecto.....	38
Ilustración 2.22 – Instalación de la biblioteca Firebase.	38
Ilustración 2.23 – Envío de datos exitoso a Firebase	41
Ilustración 2.24 – La página de la base de datos	42
Ilustración 2.25 – Almacenamiento de los valores	45
Ilustración 3.1 – Telegram Bot	50
Ilustración 3.2 – Diagrama a implementar.....	52
Ilustración 3.3 – Bot Oficial	52
Ilustración 3.4 – Apartado de creación de bot	53
Ilustración 3.5 – Agregar lista de comandos	53
Ilustración 3.6 – Librerías necesarias.....	54
Ilustración 3.7 – Configuración de red WIFI	54
Ilustración 3.8 – Definición de variables	54
Ilustración 3.9 – conexión a la red y enlace al bot.....	55
Ilustración 3.10 – comandos para los leds.	56
Ilustración 3.11 – código para sensor y sugerencias de entrada	56
Ilustración 3.12 – Resultado.....	57
Ilustración 3.13 – Sugerencias.....	57
Ilustración 4.1 - Plataforma ThingESP	62
Ilustración 4.2 - Plataforma Twilio	62
Ilustración 4.3 - Diagrama de conexión para el LED.....	64
Ilustración 4.4 - Implementación del diagrama de conexión	65
Ilustración 4.5 - Instalación de la librería ThingESP	66
Ilustración 4.6 - Creación de nuestra cuenta en ThingESP	67
Ilustración 4.7 - Registro de nuestros datos.....	67
Ilustración 4.8 - Verificación necesaria.....	68

Ilustración 4.9 - Confirmación de nuestro correo.....	68
Ilustración 4.10 - Inicio de sesion con nuestros datos	69
Ilustración 4.11 - Página de inicio de ThingESP	69
Ilustración 4.12 - Pestaña de creación del proyecto	70
Ilustración 4.13 - Creación del nombre y credenciales del Proyecto	70
Ilustración 4.14 - Proyecto creado	71
Ilustración 4.15 - Modificación del encabezado ThingESP32	71
Ilustración 4.16 - Configuración de nuestra red	72
Ilustración 4.17 - Plataforma Twilio	72
Ilustración 4.19: Registro en Twilio	73
Ilustración 4.18: Registro en Twilio	73
Ilustración 4.20 - Verificación de correo	73
Ilustración 4.21 - Ingreso de nuestro número celular	74
Ilustración 4.22 - Verificación de código SMS	74
Ilustración 4.23 - Pantalla de bienvenida.....	75
Ilustración 4.24 - Formulario en Twilio.....	75
Ilustración 4.25 - Página principal de Twilio	76
Ilustración 4.26 - Navegar hacia "Send Whatsapp message"	76
Ilustración 4.27 - Apartado Sandbox settings	77
Ilustración 4.28 - configuración del apartado "When a message comes in"	77
Ilustración 4.29 - Numero al cual le debemos enviar el mensaje y el mensaje que debe ser enviado	78
Ilustración 4.30 - Mensaje de recibido	78
Ilustración 4.31 - Funcionamiento exitoso de nuestro código.....	80
Ilustración 4.32 - Diagrama a implementar	81
Ilustración 4.33 - Funcionamiento exitoso de nuestro código.....	84

INTRODUCCIÓN

En este documento se presenta un manual de prácticas para las asignaturas “ACP-137 Introducción a IoT” y “AFDE-PPR-23 Principios de IoT”, específicamente para la Unidad IV del programa de asignatura “Aplicaciones de IoT”. Este manual incluye diversas actividades guiadas cuyo objetivo es reforzar los conocimientos del estudiantado de las carreras de Ingeniería en Sistemas de Energía e Ingeniería en Redes sobre el uso de diversas aplicaciones que se puede interconectar con una tarjeta ESP32 y así implementar un sistema IoT.

La Práctica 1 está enfocada en conocer los protocolos que han sido creados para trabajar con aplicaciones de IoT, en este caso se enfocará en el protocolo MQTT. En la segunda práctica se muestra como se puede enviar información de sensores a una base de datos usando el protocolo HTTP. La práctica 3 y 4 se enfoca en usar aplicaciones como las redes sociales de WhatsApp y Telegram para el monitoreo de variables y control de carga a distancia.

Este manual está diseñado para apoyar al profesorado en la enseñanza de la asignatura, contribuyendo así a la formación profesional del estudiantado del programa educativo de Ingeniería en Redes.

PRÁCTICA I – ESP32 CON MQTT



OBJETIVO DE LA UNIDAD:

Unidad IV: El alumno desarrollará un sistema de IoT para monitoreo de variables físicas de un proceso con almacenamiento e interacción en la nube.

OBJETIVOS DE LA PRACTICA:

Objetivo general:

- Implementar una comunicación eficiente entre un microcontrolador ESP32 y un broker MQTT público, usando sensores y actuadores, con el fin de desarrollar una base funcional para aplicaciones IoT.

ANTECEDENTES

MQTT (Message Queuing Telemetry Transport) es un protocolo de comunicación diseñado para dispositivos con pocos recursos, como los microcontroladores. Es muy útil en proyectos de Internet de las Cosas (IoT) por varias razones:

- Ligero y eficiente: Consume pocos datos, ideal para redes lentas o inestables.
- Modelo publicador/suscriptor: En lugar de enviar mensajes directos entre dispositivos, los datos se organizan por temas llamados *tópicos*.
- Un dispositivo “publica” información en un tópico (por ejemplo: `sensor/temperatura`).
- Otro dispositivo puede ****suscribirse**** a ese mismo tópico y recibir la información automáticamente.

- Broker MQTT: Es el servidor que se encarga de recibir todos los mensajes publicados y reenviarlos a los dispositivos que estén suscritos.
- Facilidad de uso: Puede usarse en redes locales o por Internet, y existen brokers públicos gratuitos como `broker.emqx.io`. (Kymera Systems, 2025)

MQTT permite que varios dispositivos se comuniquen entre sí de forma simple, sin que tengan que conocerse directamente. Como se muestra en la *Ilustración 1.1*.

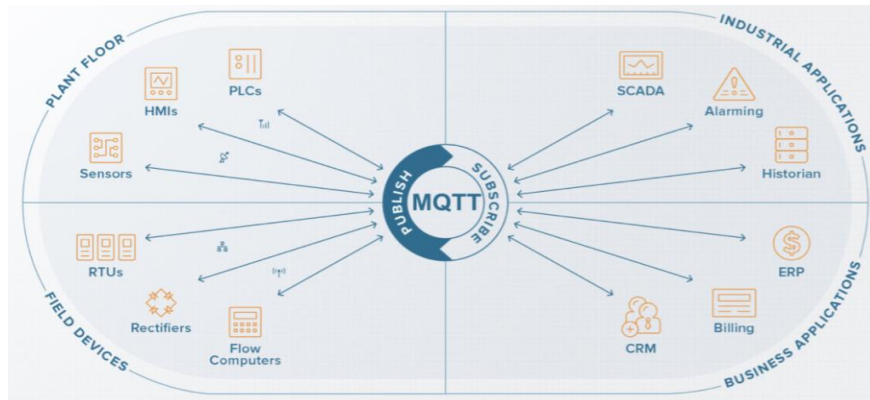


Ilustración 1.1 – Funcionamiento del bróker. (Fuente: [What is MQTT, and Why Does it Matter? | Kymera Systems Inc](#))

MATERIAL POR UTILIZAR

- ESP32, cualquier modelo
- LED de cualquier color
- Resistencia de 330Ω
- Servomotor SG90
- Sensor DHT11
- Resistencia de 4.7k
- Sensor PIR HC-SR501
- Jumpers Macho-Macho, Macho-Hembra
- Protoboard

SOFTWARE NECESARIO

- ArduinoIDE

- Broker MQTT Público: EMQX <https://www.emqx.com/en/try?product=broker>
- MIT App Inventor: <https://appinventor.mit.edu/>
-

DESARROLLO DE LA PRÁCTICA

Práctica 1. Encendido y control de cargas a través de un bróker MQTT (EMQX Public broker)

1.1. Implemente el circuito que se muestra en la *Ilustración 1.2*

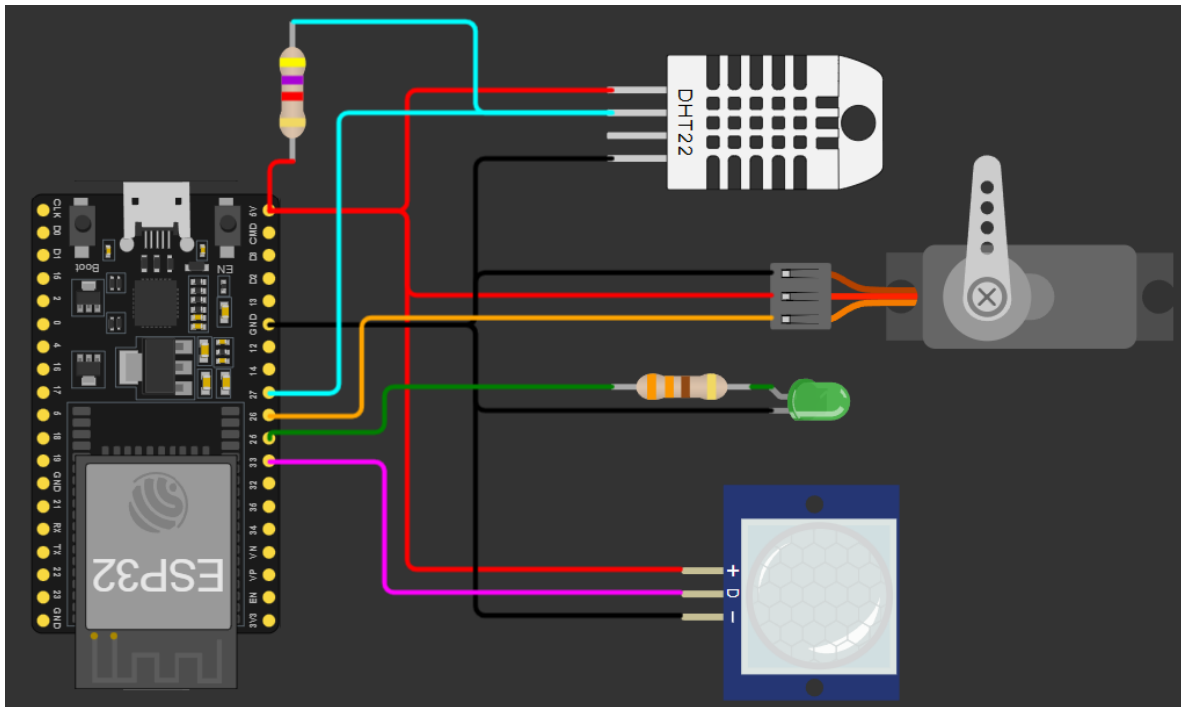


Ilustración 1.2 – Diagrama a implementar

1.2. Configuración del ESP32 en la plataforma ArduinoIDE.

Como primer paso nos dirigiremos al siguiente link: [MQTTX: Your All-in-one MQTT Client Toolbox](#) en el que se abrirá la interfaz mostrada en la *Ilustración 1.3*. Ahí seleccionamos la pestaña MQTT y se elige la opción “Free Public MQTT 5 Broker”.

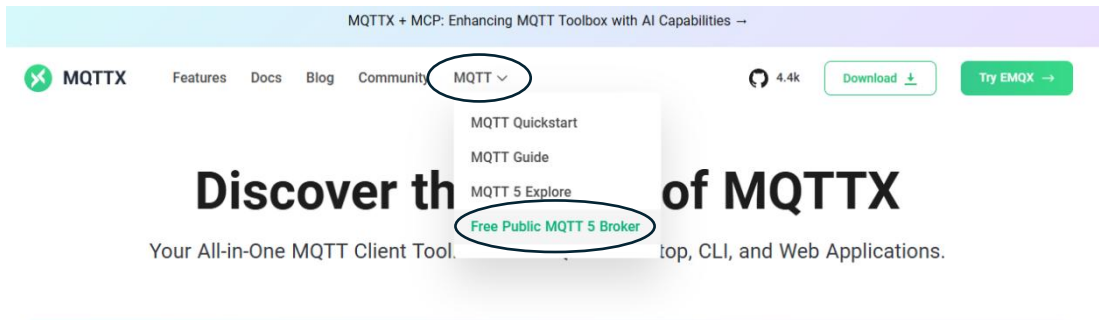


Ilustración 1.3 – Pagina del MQTTX

Se abrirá la página mostrada en la *Ilustración 1.4*. De las que obtendremos los datos Broker y TCP Port, que usaremos en la configuración del ESP32. No cerramos esta página, se usará posteriormente.

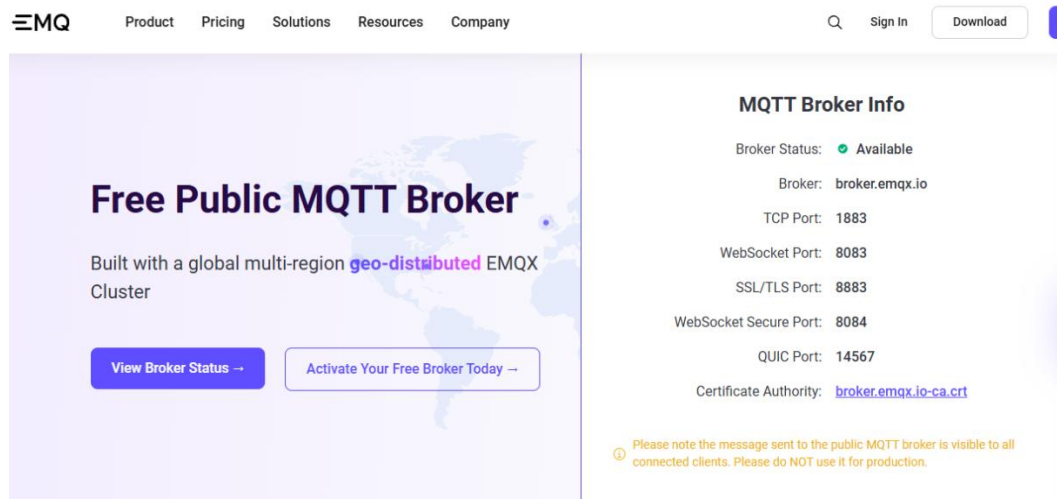


Ilustración 1.4 – Datos Broker y TCP Port.

Ahora con esos datos, procedemos a configurar y programar el ESP32 en el IDE de Arduino con el siguiente código. Los detalles y explicaciones se encuentran comentados dentro de las líneas del código.

```
#include <WiFi.h>           // Librería para conectividad WiFi
#include <PubSubClient.h>    // Librería para protocolo MQTT
#include <DHT.h>             // Librería para sensor DHT11 (temperatura/humedad)
#include <ESP32Servo.h>      // Librería para controlar servos con ESP32

// ==== Configuración de WiFi a la que se conectará el ESP32====
const char* ssid = "XXXXXXX"; // Nombre de la red WiFi
const char* password = "XXXXXX"; // Contraseña de la red WiFi

// ==== Configuración de MQTT ====
const char* mqtt_server = "broker.emqx.io"; // Aquí ingresamos el texto del broker que nos arrojó la página de la figura 1.3
const int mqtt_port = 1883; // Ingresamos el texto del TCP Port que nos arrojó la página de la figura 1.3
const char* mqtt_user = ""; // Usuario MQTT (vacío para EMQX público)
const char* mqtt_pass = ""; // Contraseña MQTT (vacío para EMQX público)

const char* client_id = "ESP32_MQTT_IoT_MANUAL"; // ID del cliente MQTT (debe ser único).
// "ESP32_MQTT_IoT_MANUAL" ID de cliente usado como ejemplo en este documento.
// Se crea un nombre con la mayor cantidad y variedad de palabras posibles.
// Esto se explica más a detalle en la sección correspondiente a la configuración del MQTT.

WiFiClient espClient; // Cliente WiFi base
PubSubClient client(espClient); // Cliente MQTT basado en WiFiClient

// ==== Definición de pines ====
#define DHT_PIN 27 // Pin donde está conectado el sensor DHT
#define SERVO_PIN 26 // Pin donde está conectado el servo
#define LED_PIN 25 // Pin donde está conectado el LED
#define PIR_PIN 33 // Pin donde está conectado el sensor de movimiento PIR

// ==== Inicialización del sensor DHT ====
#define DHTTYPE DHT11
DHT dht(DHT_PIN, DHTTYPE);

// ==== Inicialización del servo ====
Servo myServo;

// ==== Variables globales ====
long lastMsg = 0; // Temporizador para enviar datos periódicos

// ==== Conexión a WiFi ====
void setup_wifi() {
```

```

Serial.print("Conectando a WiFi");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nWiFi conectado");
Serial.print("IP local: ");
Serial.println(WiFi.localIP());
}

// ==== Publicar los valores iniciales al conectarse ====
void publicarValoresIniciales() {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    int movimiento = digitalRead(PIR_PIN);

    if (!isnan(t) && !isnan(h)) {
        String tempStr = String(t);
        String humStr = String(h);
        String movStr = (movimiento == HIGH) ? "Movimiento detectado" : "Sin movimiento";

        // Publicar en topics individuales
        client.publish("ESP32_MQTT_IoT_MANUAL/temp", tempStr.c_str());
        client.publish("ESP32_MQTT_IoT_MANUAL/hum", humStr.c_str());
        client.publish("ESP32_MQTT_IoT_MANUAL/mov", movStr.c_str());

        //ESP32_MQTT_IoT_MANUAL/temp, /hum, etc. Se usa ESP32_MQTT_IoT_MANUAL como prefijo para hacer los nombres de los
        tópicos lo más unicos posibles.

        // Publicar un resumen en JSON
        String resumen = "{\"temp\": \"" + tempStr + "\", \"hum\": \"" + humStr + "\", \"mov\": \"" + movStr + "\"}";
        client.publish("ESP32_MQTT_IoT_MANUAL/resumen", resumen.c_str());
    }
}

// ==== Callback que se ejecuta cuando llega un mensaje MQTT ====
//Esto se ejecuta cuando un cliente suscrito a algún tópico quiere manipular alguna carga en el ESP32.
void callback(char* topic, byte* payload, unsigned int length) {
    payload[length] = '\0';
    String msgStr = String((char*)payload);

    Serial.print("[MQTT] Mensaje recibido - Topic: ");
    Serial.print(topic);
    Serial.print(" | Mensaje: ");
    Serial.println(msgStr);
}

```

```

String topic_led    = String(client_id) + "/led";
String topic_servo = String(client_id) + "/servo";

if (String(topic) == topic_led) {
  if (msgStr == "ON") {
    digitalWrite(LED_PIN, HIGH);
    client.publish("ESP32_MQTT_IoT_MANUAL/led_estado", "ENCENDIDO");
    Serial.println("[Estado LED] Publicado: ENCENDIDO");
  } else {
    digitalWrite(LED_PIN, LOW);
    client.publish("ESP32_MQTT_IoT_MANUAL/led_estado", "APAGADO");
    Serial.println("[Estado LED] Publicado: APAGADO");
  }
}

if (String(topic) == topic_servo) {
  int angulo = msgStr.toInt();
  if (angulo >= 0 && angulo <= 180) {
    myServo.write(angulo);
    client.publish("ESP32_MQTT_IoT_MANUAL/servo_estado", String(angulo).c_str());
    Serial.println("[Estado SERVO] Publicado: " + String(angulo) + "°");
  }
}
}

// ==== Intentar reconectar si se pierde la conexión MQTT ====
void reconnect() {
  while (!client.connected()) {
    Serial.print("Intentando conexión MQTT...");
    if (client.connect(client_id, mqtt_user, mqtt_pass)) {
      Serial.println("Conectado al broker MQTT");

      // Suscribirse a los topics de control
      client.subscribe("ESP32_MQTT_IoT_MANUAL/led");
      client.subscribe("ESP32_MQTT_IoT_MANUAL/servo");
      //En este caso, se suscribe a esos tópicos, por que el ESP32 necesita "escuchar" los comaando que se mandan a esos
      //tópicos para poder actuar en consecuencia.
      //En los otros tópicos, el ESP32 solo necesita publicar, por eso no requiere suscribirse.

      // Publicar estado inicial
      client.publish("ESP32_MQTT_IoT_MANUAL/estado", "ESP32 conectado");
      publicarValoresIniciales();
    } else {
      Serial.print("Fallo. Estado: ");
      Serial.println(client.state());
      delay(2000);
    }
  }
}

```

```

    }
}

// ==== Configuración inicial ====
void setup() {
    Serial.begin(115200);           // Iniciar puerto serie
    pinMode(LED_PIN, OUTPUT);       // Configurar LED como salida
    pinMode(PIR_PIN, INPUT);        // Configurar sensor PIR como entrada
    dht.begin();                   // Iniciar sensor DHT
    myServo.attach(SERVO_PIN);      // Adjuntar servo al pin

    setup_wifi();                  // Conectar a WiFi
    client.setServer(mqtt_server, mqtt_port); // Configurar broker MQTT
    client.setCallback(callback);   // Definir función callback
}

// ==== Bucle principal ====
void loop() {
    if (!client.connected()) {
        reconnect();               // Reconectar si se pierde MQTT
    }
    client.loop();                 // Mantener MQTT funcionando

    long now = millis();
    if (now - lastMsg > 5000) {    // Cada 5 segundos...
        lastMsg = now;

        float t = dht.readTemperature();
        float h = dht.readHumidity();
        int movimiento = digitalRead(PIR_PIN);

        if (!isnan(t) && !isnan(h)) {
            String tempStr = String(t);
            String humStr = String(h);
            String movStr = (movimiento == HIGH) ? "Movimiento detectado" : "Sin movimiento";

            // Mostrar por Serial
            Serial.println("[MQTT] Publicando datos...");
            Serial.println("Temperatura: " + tempStr + " °C");
            Serial.println("Humedad: " + humStr + " %");
            Serial.println("Movimiento: " + movStr);

            // Publicar a topics MQTT
            client.publish("ESP32_MQTT_IoT_MANUAL/temp", tempStr.c_str());
            client.publish("ESP32_MQTT_IoT_MANUAL/hum", humStr.c_str());
        }
    }
}

```

```

    client.publish("ESP32_MQTT_IoT_MANUAL/mov", movStr.c_str());
    // La opción de publicar, se usa cuando el ESP tiene que enviar información al broker, como las señales o lecturas
    de los sensores.
    // 0 como se ve unas líneas más adelante, para actualizar el estado de las cargas.

    // Publicar resumen en JSON
    String resumen = "{\"temp\":\"" + tempStr + "\",\"hum\":\"" + humStr + "\",\"mov\":\"" + movStr + "\"}";
    client.publish("ESP32_MQTT_IoT_MANUAL/resumen", resumen.c_str());

    // Estado del LED y SERVO
    String estadoLED = (digitalRead(LED_PIN) == HIGH) ? "ENCENDIDO" : "APAGADO";
    int anguloServo = myServo.read();

    Serial.println("Estado LED: " + estadoLED);
    Serial.println("Ángulo Servo: " + String(anguloServo));

    client.publish("ESP32_MQTT_IoT_MANUAL/led_estado", estadoLED.c_str());
    client.publish("ESP32_MQTT_IoT_MANUAL/servo_estado", String(anguloServo).c_str());
}
}
}

```

En esta práctica se están utilizando 4 componentes distintos para cubrir algunas de las bases de IoT, como lo son las cargas, sensores, y PWM a través de internet. Para dejar en claro que con el uso de MQTT se puede tener control de una variedad de cosas distintas.

1.3. Configuración de MQTT

Regresamos a la página de internet que se mostraba en la *Ilustración 1.5* y bajamos hasta ver lo siguiente (ilustración 1.5), y hacemos click en “Use Online”, y se abriera una nueva pagina. (ilustración 1.6), Ahora en esta nueva página, hacemos click en crear conexión.

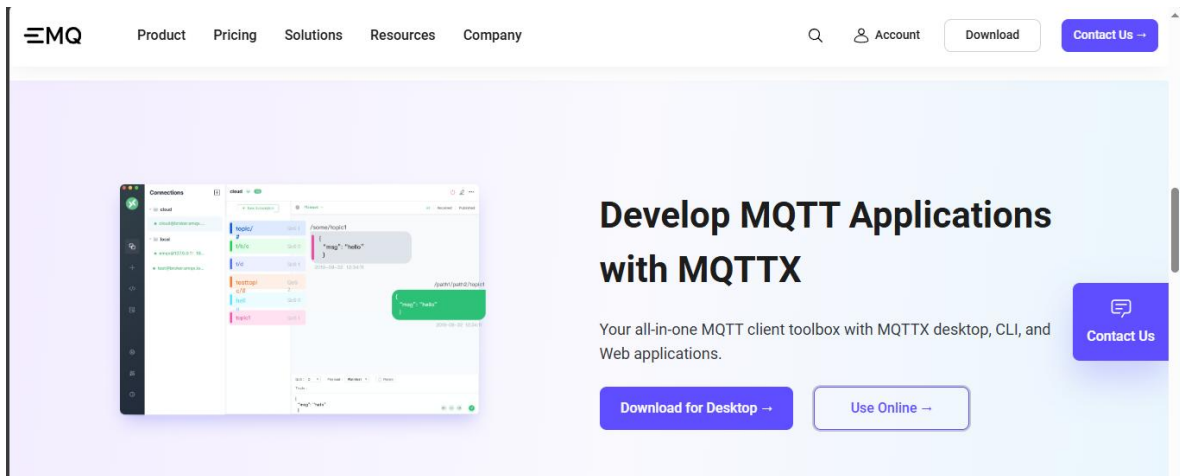


Ilustración 1.5 – Hacer click en “Use Online”

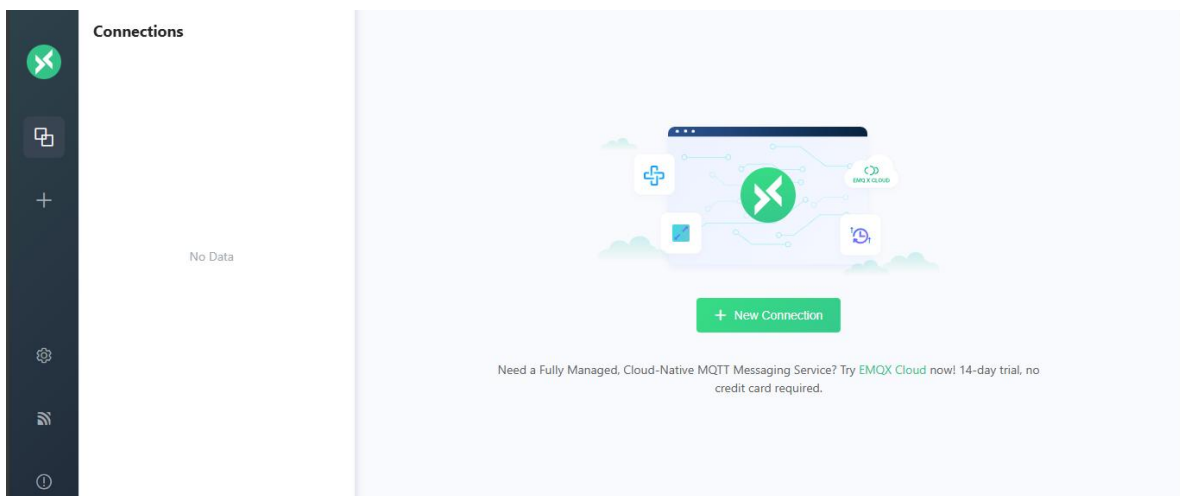
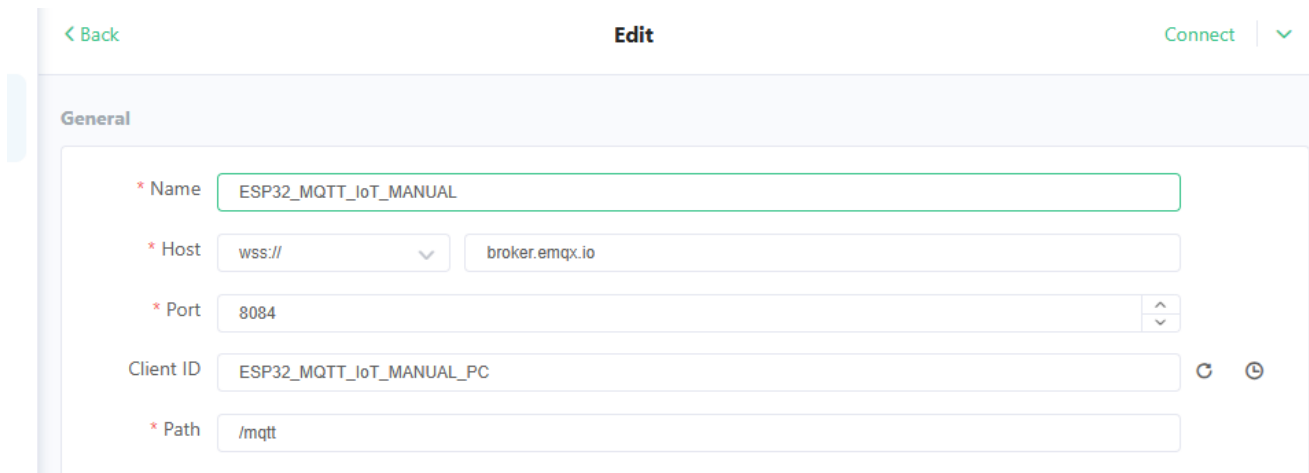


Ilustración 1.6 – Creamos la conexión

Ahora, se rellenan los espacios como se ve en la *Ilustración 1.7*.

- **Name:** Asigna un nombre a tu conexión. Este es un nombre elegido a conveniencia, no se usa realmente para establecer la conexión.
- **Host:** se deja la configuración por defecto.
- **Port:** Es el puerto que se usa para la comunicación por protocolo MQTT. En este caso se usa el 8084 por ser un navegador. El ESP32 usa el 1883.
- **Client ID:** Esta parte es importante, la razón de usar nombres con bastantes caracteres es porque el bróker no restringe nombres duplicados, lo que hace, es que si se crea una conexión con ID duplicada, desconecta automáticamente al que estaba antes. Por ejemplo, si se tiene dos clientes ESP32 con el mismo nombre, y se crea una conexión nueva con el mismo ID, el ESP32 seria desconectado automáticamente. Por esta razón se deben crear nombres muy particulares.
- **Path:** /mqtt

Una vez llenados los campos anteriores, hacemos click en Connect.



The screenshot shows a web-based configuration interface for an MQTT client. At the top, there are three buttons: "< Back" in green, "Edit" in black, and "Connect" in green with a checkmark icon. Below these is a "General" tab. The configuration fields are as follows:

- Name:** A text input field containing "ESP32_MQTT_IoT_MANUAL".
- Host:** A dropdown menu showing "wss://" and a text input field containing "broker.emqx.io".
- Port:** A numeric input field containing "8084" with up and down arrow buttons.
- Client ID:** A text input field containing "ESP32_MQTT_IoT_MANUAL_PC" with a refresh icon and a lock icon.
- Path:** A text input field containing "/mqtt".

Ilustración 1.7 – Configurando la conexión del cliente

En este paso pueden aparecer errores de conexión, sobre todo si se está usando un navegador en una PC. Estos errores pueden deberse a que los puertos requeridos no se encuentran abiertos, un antivirus o firewall bloquea el acceso, o TELNET no se encuentra activo en las configuraciones.

De haberse hecho todo lo anterior de manera correcta, se cargará una ventana (*ilustración 1.8*), indicando que le conexión fue exitosa. Seguidamente procederemos a suscribirnos a los tópicos correspondientes dando click en “New Subscription”.

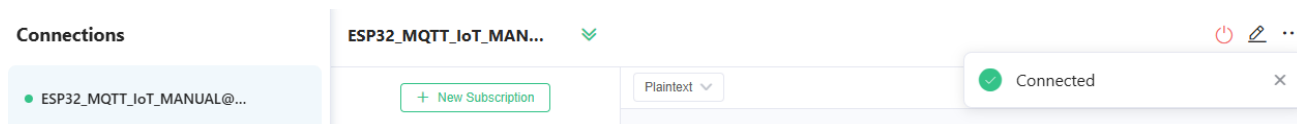


Ilustración 1.8 – Estado de conexión exitosa.

Al hacerlo se abrirá una ventana nueva, (*Ilustración 1.9*), y procederemos a llenar el recuadro Topic. Ahí es importante ingresar el nombre exacto del tópico al que se desea suscribir, exactamente el mismo tópico o tópicos que se programaron en el ESP32. Puede ser un tópico donde se escuche lo que otro cliente envía, como los tópicos dedicados a los sensores (ESP32_MQTT_IoT_MANUAL/temp, /hum, /mov) o un tópico para dar órdenes a un controlador como el ESP32 (ESP32_MQTT_IoT_MANUAL/led, /servo). De igual manera pueden ser tópicos dedicados a informar sobre el estado de las cargas (ESP32_MQTT_IoT_MANUAL/led_estado, /servo_estado). La finalidad del tópico no hace alguna diferencia al momento de suscribirse.

De igual manera, es importante que los nombres sean únicos, ya que de haber 2 iguales, podrías suscribirte a cualquiera de los dos sin posibilidad de elegir. En este caso el bróker no elimina o desconecta el tópico más antiguo, los deja existir.

Podemos suscribirnos individualmente a cada tópico creando múltiples suscripciones e ingresando sus nombres completos:

```
ESP32_MQTT_IoT_MANUAL/temp
ESP32_MQTT_IoT_MANUAL/hum
ESP32_MQTT_IoT_MANUAL/led
ESP32_MQTT_IoT_MANUAL/led_estado
ESP32_MQTT_IoT_MANUAL/servo
ESP32_MQTT_IoT_MANUAL/servo_estado
ESP32_MQTT_IoT_MANUAL/mov
```

Y tendríamos 7 ventanas, una para cada tópico.

En este caso, haremos una suscripción general usando el sufijo “/#” después de ESP32_MQTT_IoT_MANUAL, esto es para suscribirnos a todos los tópicos que inicien con ESP32_MQTT_IoT_MANUAL. Y tendríamos toda la información de los 7 tópicos mencionados anteriormente, como se muestra en la *Ilustración 1.9*.

The image shows a 'New Subscription' dialog box. The 'Topic' field is filled with 'testtopic/#'. The 'QoS' field is set to '0' and has a dropdown menu showing 'At most once'. The 'Color' field is set to '#7EB215' and has a color picker icon. The 'Alias' field is empty. The 'Subscription Identifier' field is empty. The 'No Local' field has two radio buttons, 'true' and 'false'. The 'Retain as Published' field has two radio buttons, 'true' and 'false'. The 'Retain Handling' field has a dropdown menu showing 'Select'. At the bottom right are 'Cancel' and 'Confirm' buttons.

Ilustración 1.9 – Suscribiéndose a un tópico

Si todo se realizó de manera correcta, estaremos suscritos a los tópicos correspondientes y se podrá ver la información enviada por nuestro ESP32. (*Ilustración 1.10*, recuadros grises) Y es en ese mismo espacio en donde podremos dar órdenes a nuestro ESP. En la esquina inferior derecha.

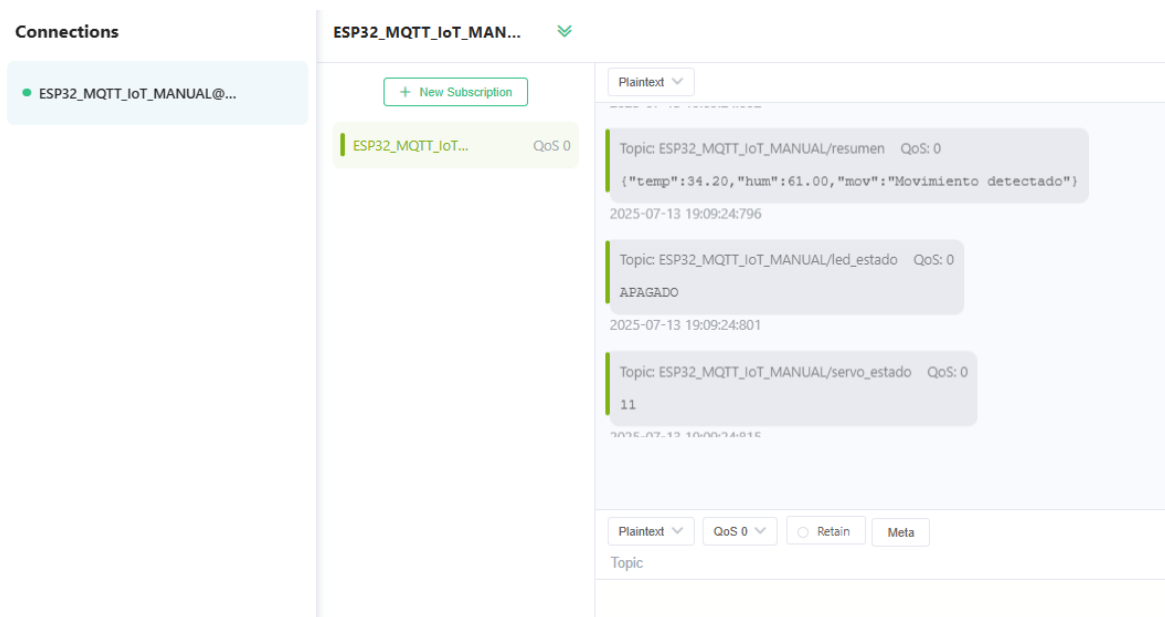


Ilustración 1.10 – Visualizando lo que envía la ESP32 a los tópicos.

Para interactuar con un tópico, nos dirigimos a la esquina inferior derecha, y procuramos que la primera pestaña tenga seleccionado “Plaintext”. Plaintext envía la información en texto plano, pero también tiene la opción de enviarlos en formato json.

En donde dice “Topic”, escribiremos el tópico exacto en el que queremos publicar, en nuestro caso solo son ESP32_MQTT_IoT_MANUAL/led o ESP32_MQTT_IoT_MANUAL/servo. Todos los demás tópicos sirven solo para lectura y publicar en ellos no produce nada. Por último, debajo de Topic, es donde escribimos nuestro mensaje según lo que hayamos programado en el ESP32. Y le damos enviar. Encenderemos el LED, *Ilustración 1.11*.

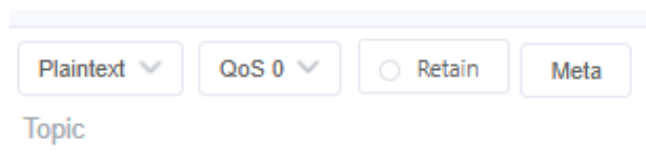


Ilustración 1.11 – Espacio donde publicamos en los tópicos

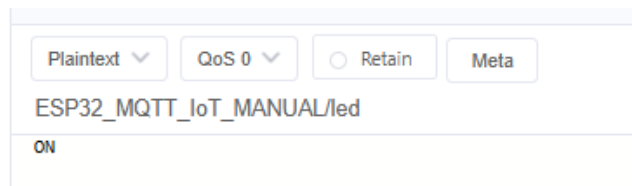


Ilustración 1.12 – A la espera de enviar el comando

Una vez enviado, podremos ver en la ventana el comando enviado y el estado de nuestro LED.
Ilustración 1.13.

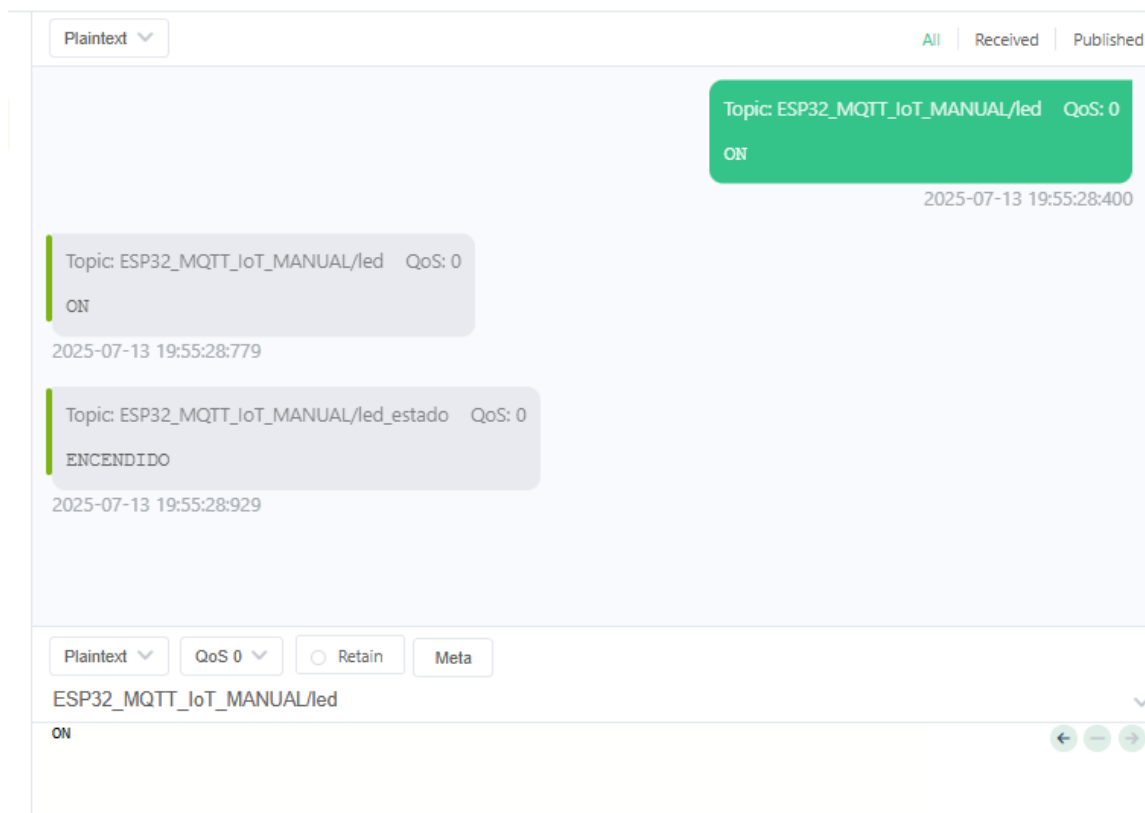


Ilustración 1.13 – Comando enviado y nuevo estado del LED

1.4. Aplicación Android para control y monitoreo

Como parte del sistema IoT, se puede desarrollar una aplicación móvil usando la plataforma MIT App Inventor.

Esta app permite al usuario:

- Conectarse o desconectarse del broker MQTT.
- Encender y apagar un LED de manera remota.
- Visualizar en tiempo real la temperatura y humedad ambiente.
- Detectar movimiento por medio del sensor PIR.
- Controlar el ángulo de un servomotor desde la interfaz gráfica.

La interfaz está compuesta por:

- Un interruptor (Switch) llamado BROKER, que permite establecer o finalizar la conexión MQTT.
- Un botón llamado LED, que alterna el estado del LED.
- Etiquetas TEMPERATURA, HUMEDAD y MOVIMIENTO, que muestran en tiempo real los datos enviados por el ESP32.
- Un slider llamado SERVO_SLIDER, que permite controlar el ángulo del servo.
- Una etiqueta SERVO_DISPLAY, que muestra el ángulo actual del servo.

Esta app se comunica con el ESP32 usando el protocolo MQTT, enviando comandos y recibiendo información a través de tópicos específicos. Su propósito es ofrecer una interfaz intuitiva y funcional para la interacción remota con el sistema físico. (*Ilustración 1.14*).



Ilustración 1.14 – App que permite la visualización y control de los tópicos de manera más ordenada y sencilla.

Archivo del proyecto App Inventor

Para quienes deseen revisar, modificar o aprender de la aplicación móvil desarrollada en esta práctica, se proporciona el archivo fuente del proyecto en formato `.aia`, compatible con MIT App Inventor.

Puede descargarse desde el siguiente enlace:

<https://drive.google.com/file/d/1Op8IkAanncwNX7iLwi5Fjy8s5tqCVOMK/view?usp=drivesdk>

Este archivo puede abrirse directamente desde <https://appinventor.mit.edu/>

mediante la opción "Importar un proyecto.aia desde mi computadora".

> Nota: No se incluye la explicación detallada de los bloques, ya que estos pueden analizarse visualmente una vez cargado el proyecto en App Inventor.

Justificación de la extensión MQTT para App Inventor

MIT App Inventor, en su forma original, no incluye soporte nativo para el protocolo MQTT. Por esta razón, fue necesario recurrir al uso de una extensión externa: `com.sprijatelj.mqtt`, comúnmente conocida como: "UrsPahoMqttClient". Se muestra en el área de componentes invisibles, donde van el Bluetooth, alertas, etc

Esta extensión se basa en la librería "Eclipse Paho", ampliamente utilizada en el ecosistema MQTT, y fue diseñada para ser compatible con App Inventor y sus derivados (Kodular, Niotron, etc.). Permite establecer una conexión entre la aplicación móvil y un **broker MQTT**, posibilitando el envío y recepción de mensajes de forma eficiente y ligera.

- El protocolo MQTT es fundamental en IoT (Internet of Things) por su bajo consumo de ancho de banda, eficiencia y arquitectura basada en eventos.
- App Inventor no posee bloques integrados para trabajar con este protocolo.
- La extensión UrsPahoMqttClient añade componentes visuales y bloques que permiten:
 - Conectarse a un broker.
 - Suscribirse a tópicos.
 - Publicar mensajes.
 - Detectar eventos como recepción de mensajes, conexión, desconexión, etc.

Beneficios educativos de usar la extensión

- Facilita el aprendizaje de conceptos como cliente-servidor, tópicos, y eventos asíncronos.
- Permite el control de hardware real (como el ESP32) desde una app móvil, con pocos bloques.
- Fomenta la modularidad y el pensamiento orientado a eventos, ya que se trabaja con "respuestas" a mensajes.

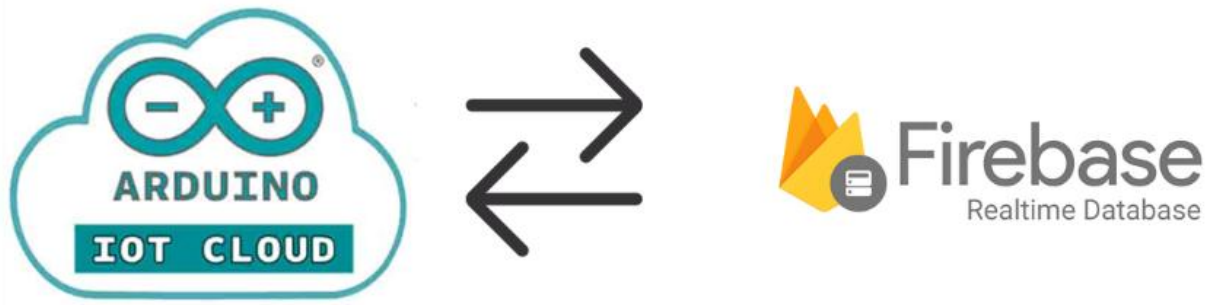
Consideraciones técnicas

- La extensión fue descargada desde fuentes confiables, como [App Inventor Extensions] (<https://puravidaapps.com/extensions.php>) o [community.kodular.io](<https://community.kodular.io/>).
- El archivo tiene extensión `.aix` y fue cargado al proyecto desde el menú de "Extensiones" > "Importar extensión desde archivo .aix".
- En resumen, el uso de esta extensión fue esencial para dotar al proyecto de capacidades IoT modernas, sin abandonar la sencillez visual que caracteriza a App Inventor.

Conclusiones

[illegible]

PRÁCTICA II – ESP32 Y CONEXIÓN A UNA BASE DE DATOS



OBJETIVO DE LA UNIDAD:

Unidad IV: El alumno desarrollará un sistema de IoT para monitoreo de variables físicas de un proceso con almacenamiento e interacción en la nube.

OBJETIVOS DE LA PRACTICA:

Conectar la ESP32 a una base de datos de Firebase para mediciones de temperatura y humedad, para su posterior análisis.

ANTECEDENTES

Si se requiere conectar tarjetas de desarrollo como Arduino, ESP32, Rasberry Pi o alguna de las tarjetas de desarrollo más comunes a Internet, para el envío de información, una de las maneras más sencillas es empleando la plataforma online Arduino IoT Cloud, *Ilustración 2.1*.

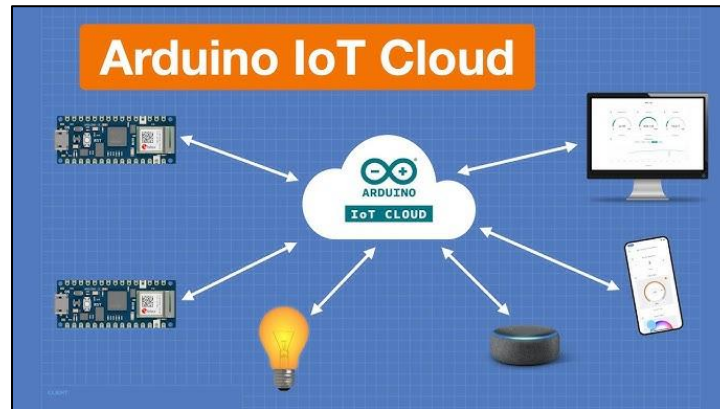


Ilustración 2.1 – Arduino IoT Cloud

Los usos que se le pueden dar a esta plataforma son:

- **Monitorización** de datos: es posible monitorizar fácilmente los valores de los sensores de Arduino a través de un panel de control.
- **Sincronización** de variables: la sincronización de variables nos permite sincronizar variables entre dispositivos, permitiendo la comunicación entre dispositivos con una codificación mínima.
- **Programación**: podemos programar trabajos para que se activen/desactiven diferentes funciones durante una cantidad de tiempo específica (segundos, minutos, horas).
- **Cargas Over-The-Air (OTA)**: permite cargar nuestro código a dispositivos no conectados al ordenador.
- **Webhooks**: integra tu proyecto con otro servicio, como IFTTT.
- **Soporte de Amazon Alexa**: permite que nuestro proyecto sea controlado por voz con la integración de Amazon.
- **Dashboard Sharing**: comparte tus datos con otras personas de todo el mundo (Söderby, 2025)

MATERIAL POR UTILIZAR

- ## SOFTWARE NECESARIO

SOFTWARE NECESARIO

IoT Cloud: <https://cloud.arduino.cc/>

Firestore: <https://firebase.google.com/?hl=es-419>

DESARROLLO DE LA PRÁCTICA

Práctica 2. Almacenamiento en una base de datos.

2.1 Implementar el circuito del diagrama mostrado en la *Ilustración 2.2*.

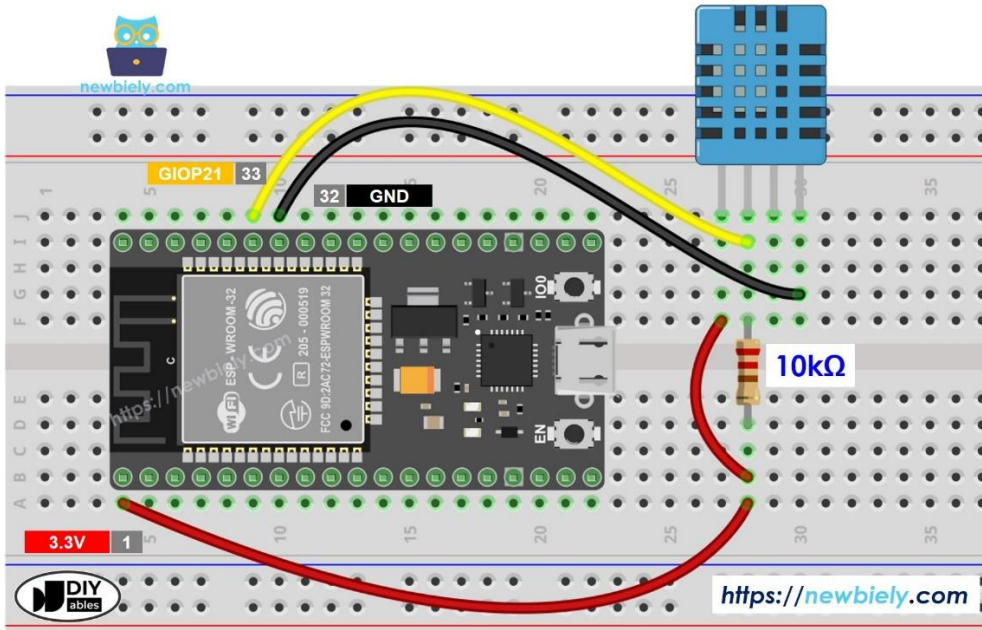
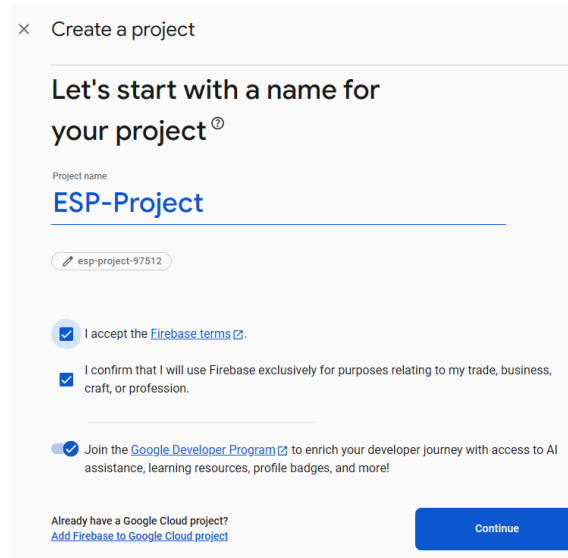


Ilustración 2.2 – Diagrama a implementar

2.2 Configuración de una cuenta de Firebase y crear un nuevo proyecto

Siga las siguientes instrucciones para crear un nuevo proyecto en Firebase:

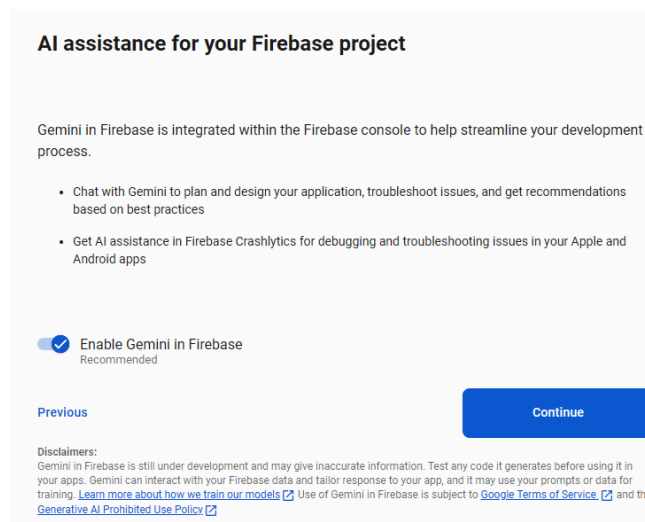
Vaya a [Firebase](#) e inicie sesión con una cuenta de Google. Vaya a la [consola Firebase](#) y cree un nuevo proyecto. Elige un nombre para tu proyecto, por ejemplo: ESP-Project, y haz clic en **Continue**, *Ilustración 2.3*.



The screenshot shows the 'Create a project' dialog in the Firebase console. At the top, it says 'Let's start with a name for your project'. Below this, the 'Project name' field is filled with 'ESP-Project'. Underneath, there is a text input field containing 'esp-project-97512'. There are three checkboxes, all of which are checked: 'I accept the [Firebase terms](#)', 'I confirm that I will use Firebase exclusively for purposes relating to my trade, business, craft, or profession.', and 'Join the [Google Developer Program](#) to enrich your developer journey with access to AI assistance, learning resources, profile badges, and more!'. At the bottom left, there is a link 'Already have a Google Cloud project? [Add Firebase to Google Cloud project](#)'. At the bottom right, there is a blue 'Continue' button.

Ilustración 2.3 – Pestaña de creación de un proyecto

A continuación, active o desactive la asistencia de IA para tu proyecto. Esto es opcional, *Ilustración 2.4*.



The screenshot shows the 'AI assistance for your Firebase project' dialog. It starts with the title 'AI assistance for your Firebase project'. Below the title, it says 'Gemini in Firebase is integrated within the Firebase console to help streamline your development process.' There are two bullet points: 'Chat with Gemini to plan and design your application, troubleshoot issues, and get recommendations based on best practices' and 'Get AI assistance in Firebase Crashlytics for debugging and troubleshooting issues in your Apple and Android apps'. Below the bullet points, there is a toggle switch labeled 'Enable Gemini in Firebase' with the word 'Recommended' underneath it. The toggle is currently turned on. At the bottom left, there is a link 'Previous'. At the bottom right, there is a blue 'Continue' button. At the very bottom, there is a 'Disclaimers' section with small text: 'Gemini in Firebase is still under development and may give inaccurate information. Test any code it generates before using it in your apps. Gemini can interact with your Firebase data and tailor response to your app, and it may use your prompts or data for training. [Learn more about how we train our models](#) Use of Gemini in Firebase is subject to [Google Terms of Service](#) and the [Generative AI Prohibited Use Policy](#)'.

Ilustración 2.4 – Pestaña para desactivar la asistencia de IA

Desactive la opción *"Habilitar Google Analytics para este proyecto"*, ya que no es necesaria. A continuación, haga clic en **"Create project"**, *Ilustración 2.5*.

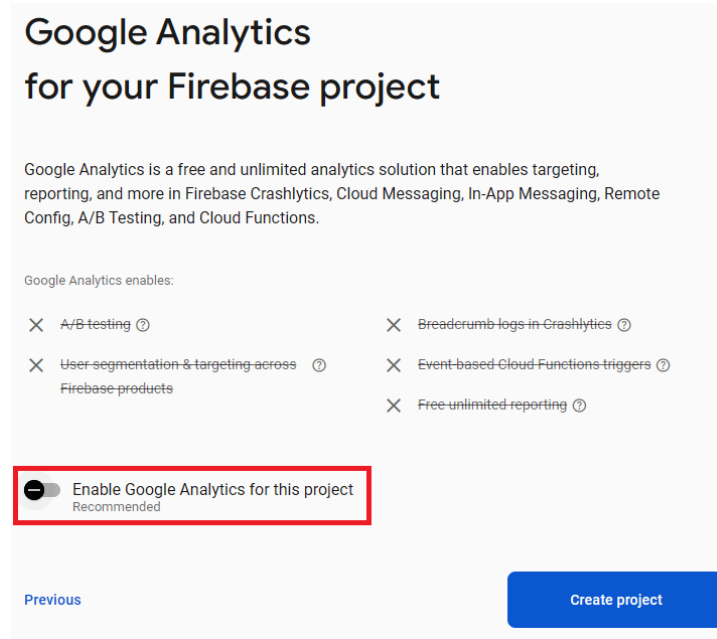


Ilustración 2.5 – Pestaña para habilitar Google Analytics

La configuración del proyecto tardará unos segundos. Haz clic en **"Continue"** cuando esté listo, *Ilustración 2.6*.

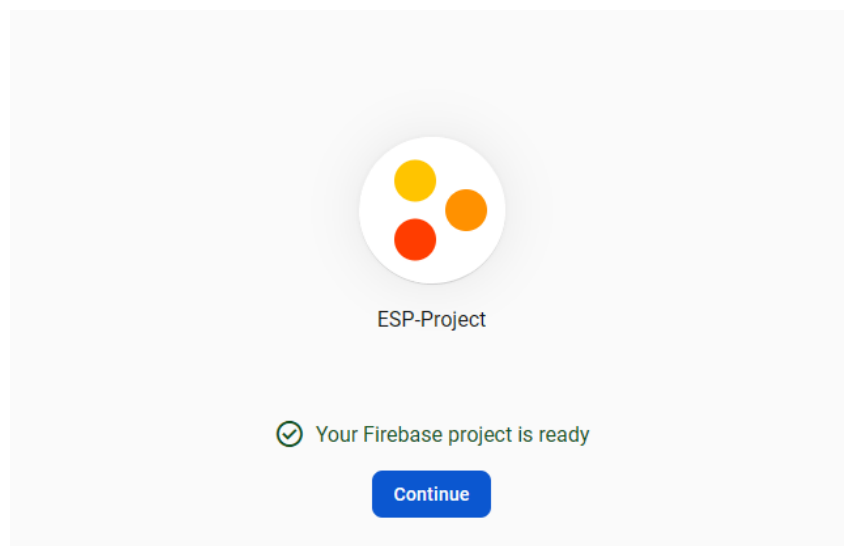


Ilustración 2.6 – Cargando la configuración del proyecto

Serás redirigido a la página de la consola de tu proyecto, *Ilustración 2.7*.

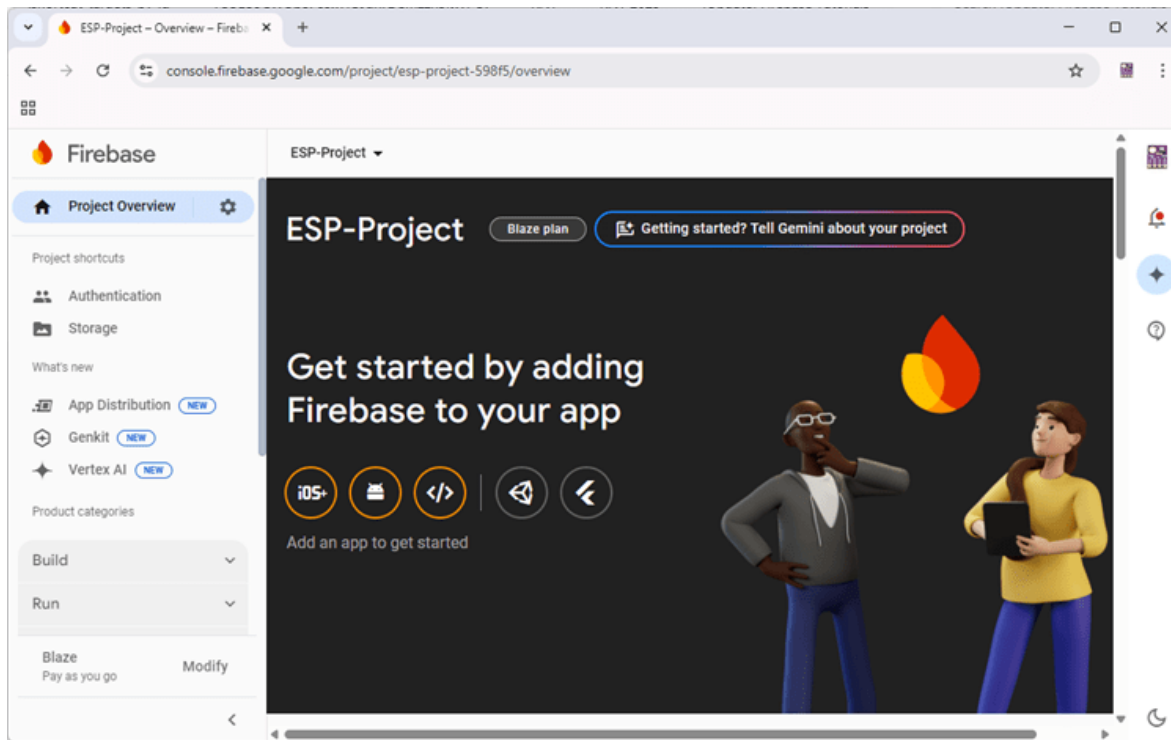


Ilustración 2.7 – Pantalla de inicio de la página de consola

Estableciendo los métodos de autenticación

Es necesario que la aplicación conozca nuestra identidad como usuario. En otras palabras, se encarga de iniciar sesión e identificar a los usuarios (en este caso, el ESP32). Conocer la identidad del usuario permite a la aplicación guardar de forma segura sus datos en la nube.

Siga las siguientes instrucciones para configurar la autenticación:

En la barra lateral izquierda, haga clic en **Build > Authentication** y luego en **Get started**, *Ilustración 2.8*.

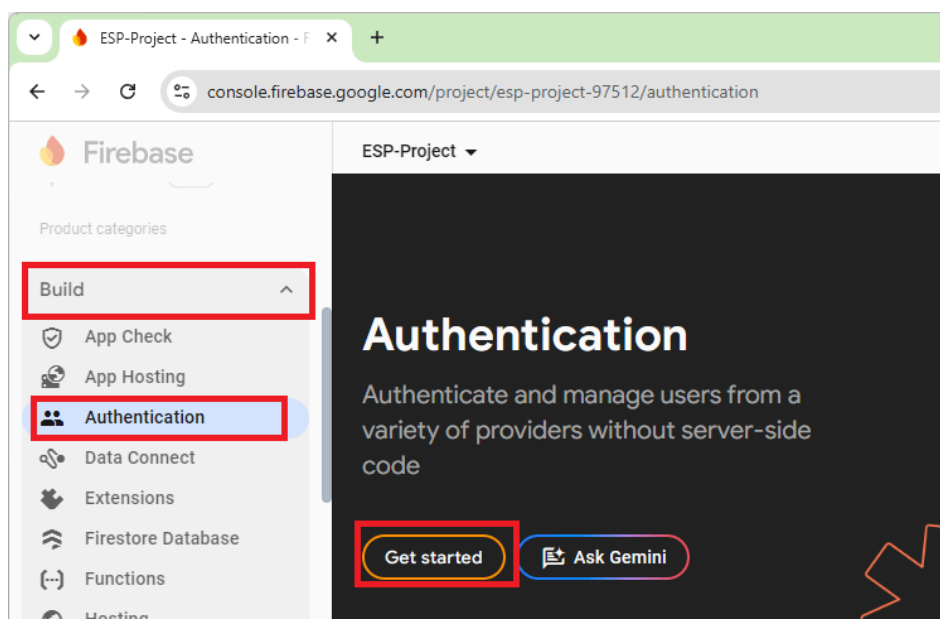


Ilustración 2.8 – Pestaña para autenticación

Existen varios métodos de autenticación, como correo electrónico y contraseña, cuenta de Google, cuenta de Facebook y otros, *Ilustración 2.9*.

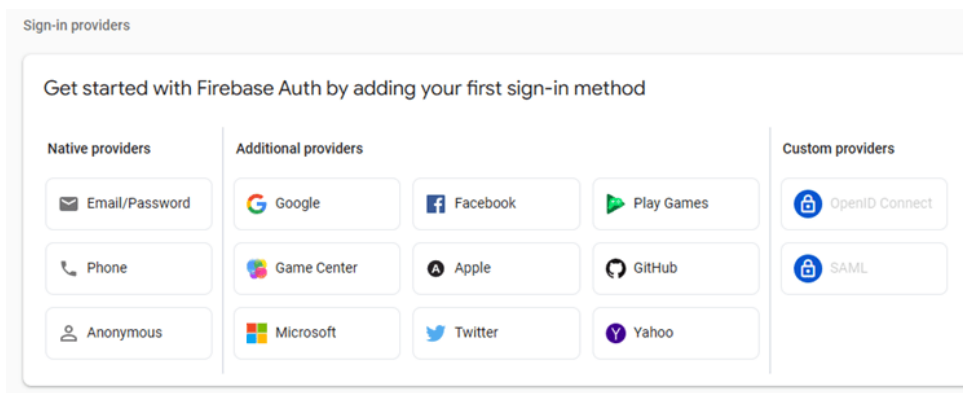


Ilustración 2.9 – Métodos para autenticarte en la aplicación

Seleccione **Email/Password** y habilite ese método de autenticación. Luego, haga clic en **Save**, *Ilustración 2.10*.

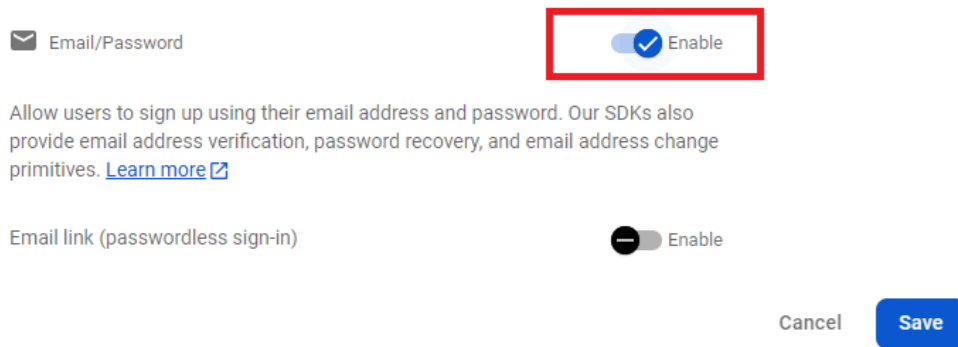


Ilustración 2.10 – Vincular la cuenta y guardar la configuración

Luego, en la parte superior, haz clic en la pestaña **Users**. Después, haz clic en **Add user**, *Ilustración 2.11*.

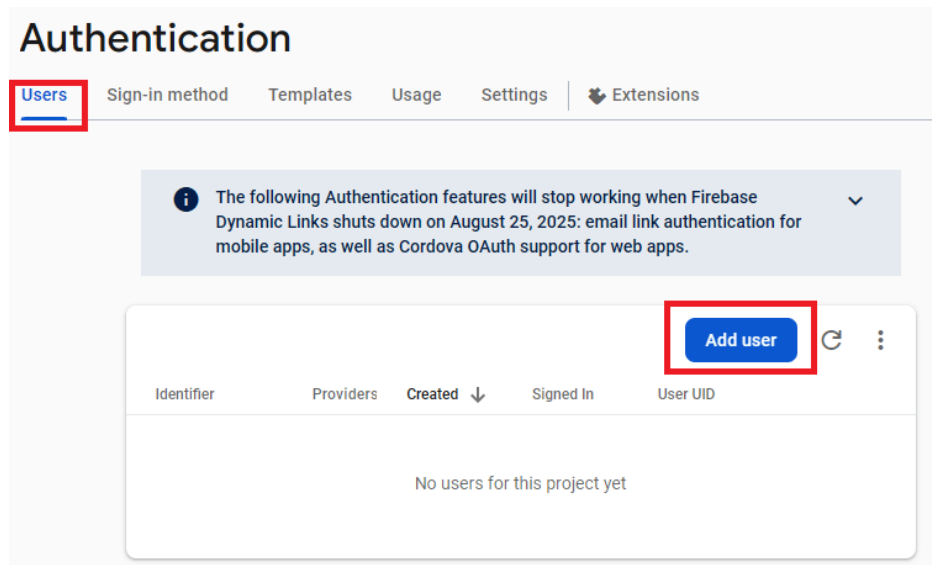
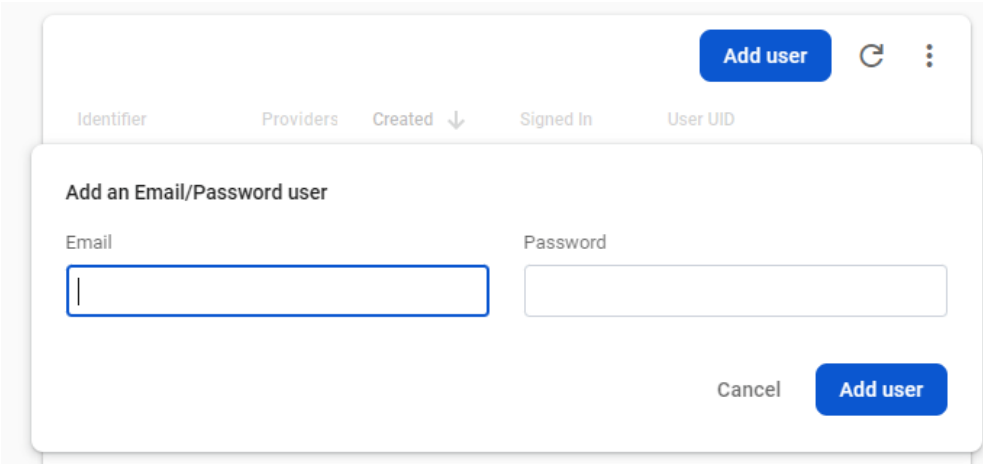


Ilustración 2.11 – Apartado para agregar un usuario

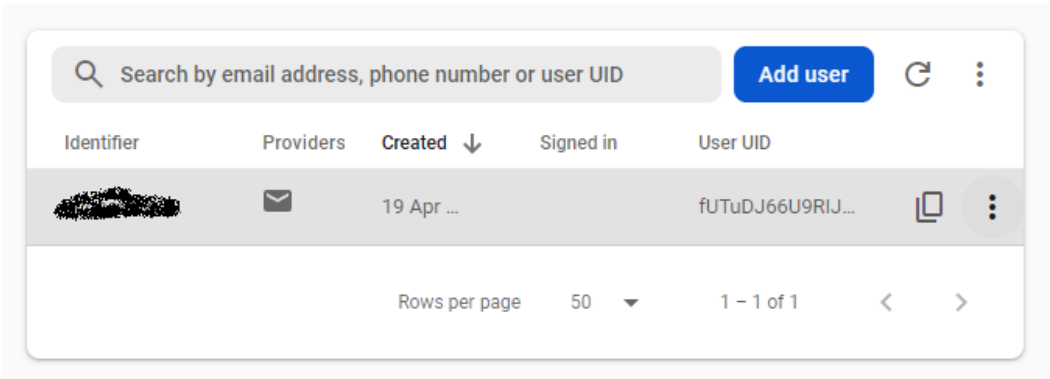
Cree un nuevo usuario con un correo electrónico y una contraseña. Puede usar su correo personal. Cree una contraseña para ese usuario (necesitará recordarla más adelante). Finalmente, haz clic en " **Add user**", *Ilustración 2.12*.



The screenshot shows a modal window titled "Add an Email/Password user". It contains two input fields: "Email" and "Password". Below the fields are two buttons: "Cancel" and "Add user". The modal is overlaid on a table with columns: Identifier, Providers, Created, Signed in, and User UID. In the top right corner of the modal, there is an "Add user" button, a refresh icon, and a menu icon.

Ilustración 2.12 – Apartado para registrar un usuario

El usuario aparecerá en la lista de usuarios. Podrás ver información sobre él, como su fecha de creación, la última vez que inició sesión y su UID, *Ilustración 2.13*.



The screenshot shows a table with the following columns: Identifier, Providers, Created, Signed in, and User UID. There is a search bar at the top with the text "Search by email address, phone number or user UID" and an "Add user" button. The table contains one row with the following data: Identifier (a blurred image), Providers (an envelope icon), Created (19 Apr ...), Signed in (empty), and User UID (fUTuDJ66U9RIJ...). Below the table, there is a pagination bar with "Rows per page" set to 50 and "1 - 1 of 1".

Identifier	Providers	Created	Signed in	User UID
		19 Apr ...		fUTuDJ66U9RIJ...

Ilustración 2.13 – Lista de usuarios

Creación de una base de datos en tiempo real

A continuación, se crea una base de datos en tiempo real para realizar la práctica. Siga los pasos a continuación para crearla.

En la barra lateral izquierda, haga clic en **Build > Realtime Database** y luego, haga clic en **Create Database**, *Ilustración 2.14*.

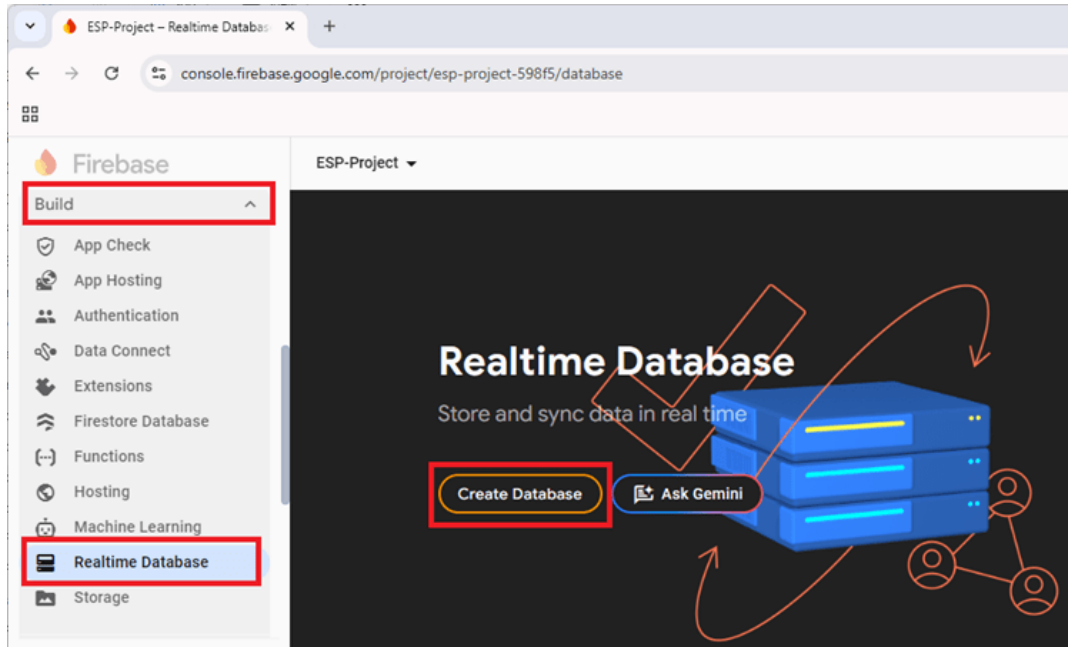


Ilustración 2.14 – Pestaña para crear la base de datos

Selecciona la ubicación de la base de datos. Debe ser la más cercana a su ubicación, *Ilustración 2.15*.

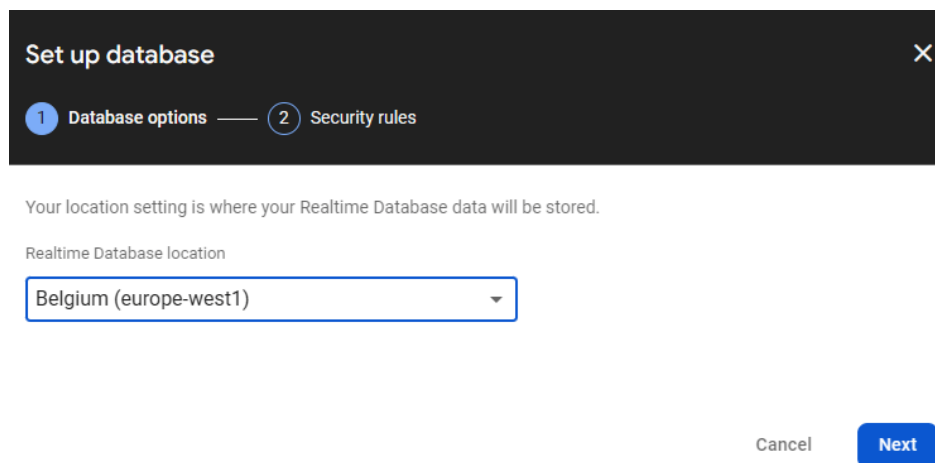


Ilustración 2.15 – Seleccionar la ubicación de la base datos

Configure reglas de seguridad para su base de datos. Para realizar pruebas, seleccione " **Start in Test mode**" y después darle clic **Enable** permitir la configuración, *Ilustración 2.16*.

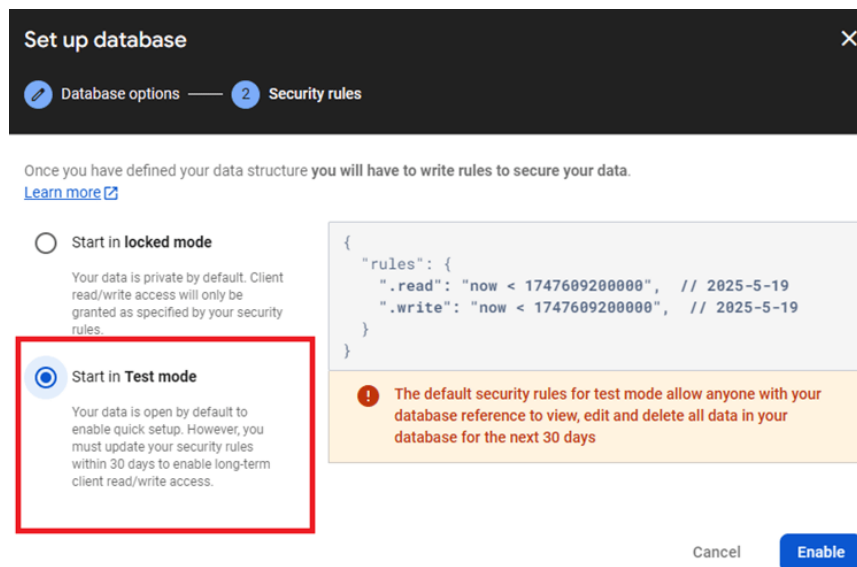


Ilustración 2.16 – configuración de las reglas de seguridad

La base de datos ya está creada. Ahora copie y guarde la URL de la base de datos (resaltada en la siguiente imagen), ya que se necesitará más adelante en el código de la ESP32, *Ilustración 2.17*.

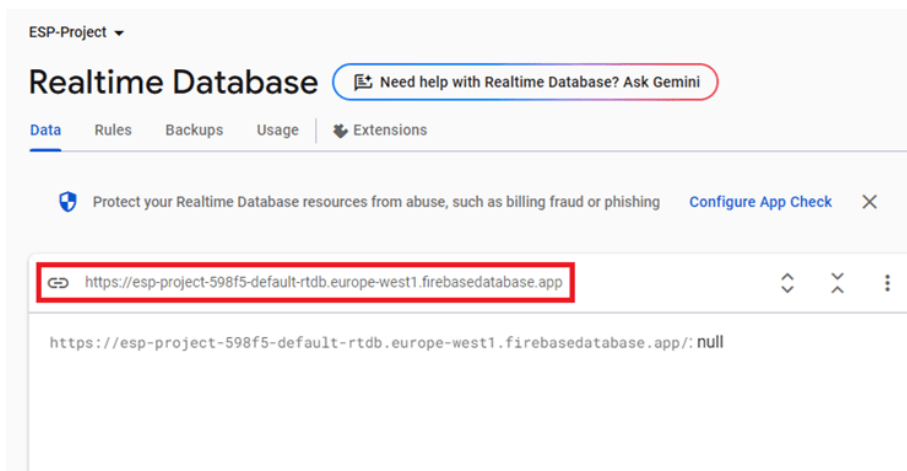


Ilustración 2.17 – Copiar la URL de la base de datos

La base de datos en tiempo real está lista. Ahora se requiere obtener la clave API del proyecto.

2.3 Obtener la clave API del proyecto

Para obtener la clave API del proyecto, en la barra lateral izquierda, haga clic en **Project settings**, *Ilustración 2.18*.

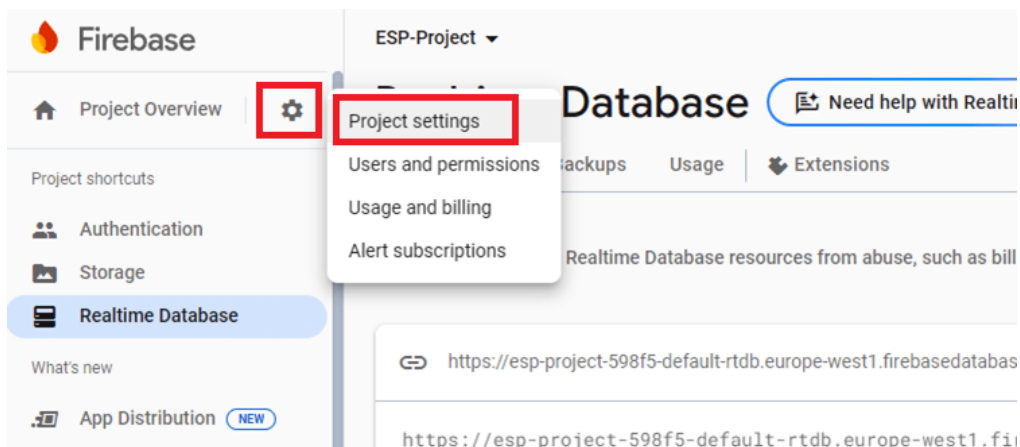


Ilustración 2.18 – Apartado de la configuración del proyecto

Copia la clave API en un lugar seguro porque la necesitarás más adelante, *Ilustración 2.19*.

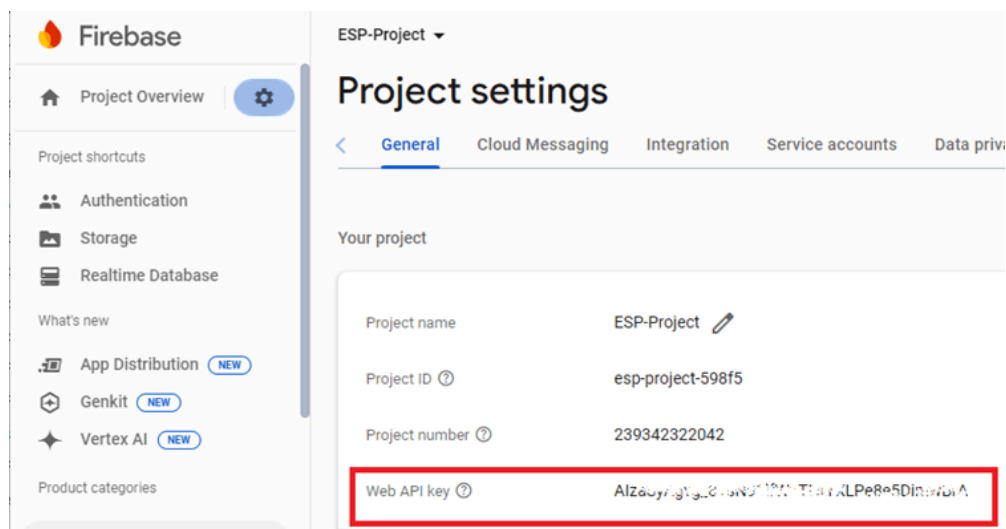


Ilustración 2.19 – Copiar la clave de la API

Ahora ya se tiene todo listo para interconectar la ESP32 con la base de datos.

Para programar el ESP32, vamos a utilizar **VS Code** con la extensión **PlatformIO**.

2.4 Instalación de la biblioteca FirebaseClient

Una vez dentro VS Code con la extensión PlatformIO, haga clic en el icono de **PIO Home** y seleccione la **Libraries**. Busque " **FirebaseClient** ". Seleccione la **biblioteca Firebase Client** de Mobitz, *Ilustración 2.20*.

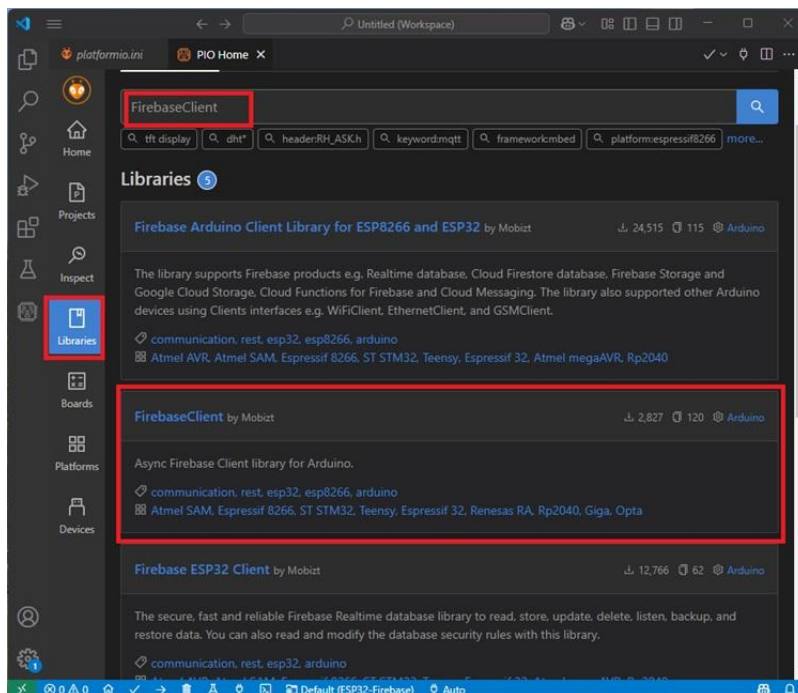
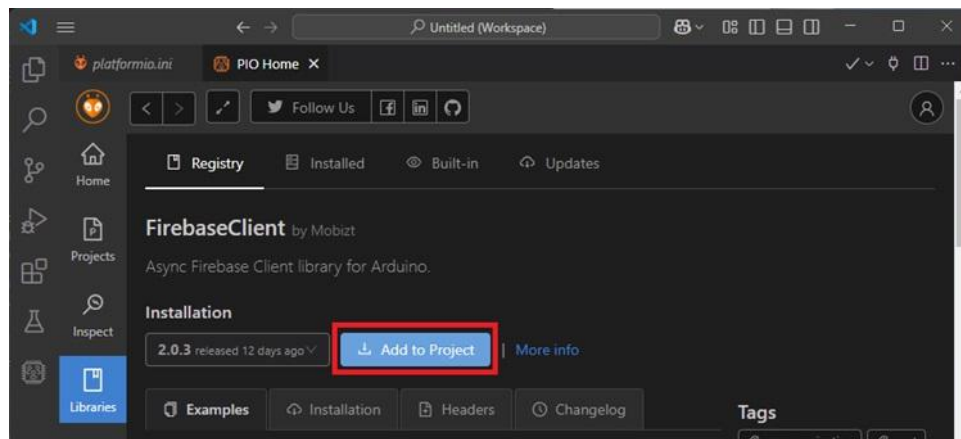


Ilustración 2.20– Configuración de la biblioteca

Luego, haga clic en **Add to Project** y seleccione el proyecto en el que está trabajando, *Ilustración 2.21*.



De igual manera, cambie la velocidad del monitor a 115200 agregando la siguiente línea *alplatformio.ini*archivo a su proyecto:

```
monitor_speed = 115200
```

2.5 Instalación de la biblioteca FirebaseClient

Dentro del Arduino IDE, siga los siguientes pasos para instalar la biblioteca.

1. Vaya a **Sketch > Incluir biblioteca > Administrar bibliotecas**
2. Busca **FirebaseClient** e instálalo de **Mobitz** . Usamos la versión 2.0.3, *Ilustración 2.22*.

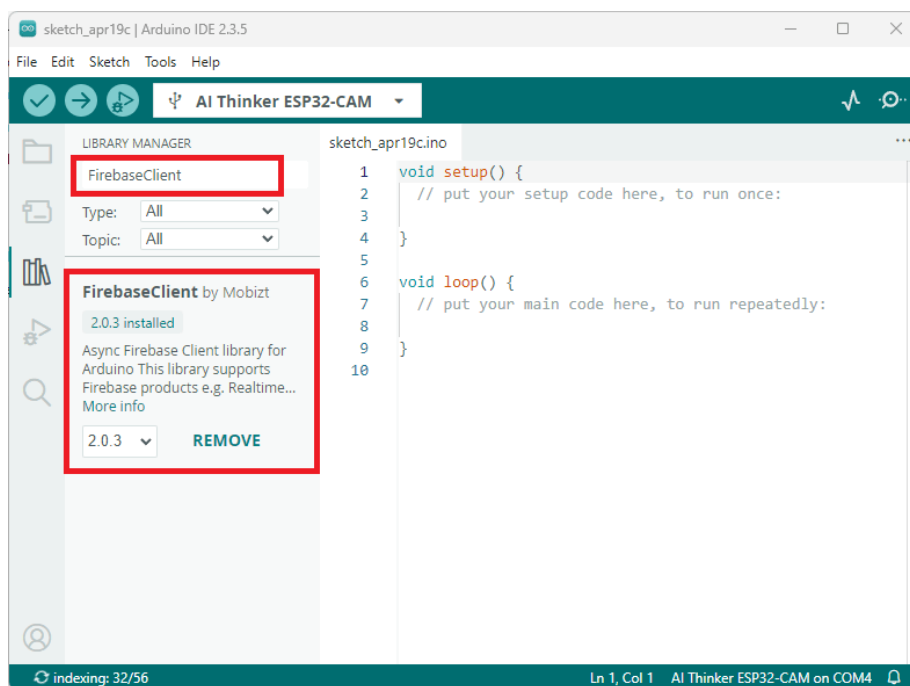


Ilustración 2.22 – Instalación de la biblioteca Firebase.

Ahora ya está todo listo para comenzar a programar la placa ESP32 e interactuar con la base de datos.

2.6 Almacenar los datos en la base de datos de Firebase

Copia el siguiente código a su IDE de Arduino. Este programa inserta un entero, un número flotante y una cadena en la base de datos cada 10 segundos. Este es un ejemplo sencillo que muestra cómo conectar el ESP32 a la base de datos para almacenar datos.

```

/*****
  Rui Santos & Sara Santos - Random Nerd Tutorials
  Complete instructions at https://RandomNerdTutorials.com/esp32-firebase-realtime-database/
  *****/
#define ENABLE_USER_AUTH
#define ENABLE_DATABASE

#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <FirebaseClient.h>

// Network and Firebase credentials
#define WIFI_SSID "REPLACE_WITH_YOUR_SSID"
#define WIFI_PASSWORD "REPLACE_WITH_YOUR_PASSWORD"

#define Web_API_KEY "REPLACE_WITH_YOUR_FIREBASE_PROJECT_API_KEY"
#define DATABASE_URL "REPLACE_WITH_YOUR_FIREBASE_DATABASE_URL"
#define USER_EMAIL "REPLACE_WITH_FIREBASE_PROJECT_EMAIL_USER"
#define USER_PASS "REPLACE_WITH_FIREBASE_PROJECT_USER_PASS"

// User function
void processData(AsyncResult &aResult);

// Authentication
UserAuth user_auth(Web_API_KEY, USER_EMAIL, USER_PASS);

// Firebase components
FirebaseApp app;
WiFiClientSecure ssl_client;
using AsyncClient = AsyncClientClass;
AsyncClient aClient(ssl_client);
RealtimeDatabase Database;

// Timer variables for sending data every 10 seconds
unsigned long lastSendTime = 0;
const unsigned long sendInterval = 10000; // 10 seconds in milliseconds

// Variables to send to the database
int intValue = 0;
float floatValue = 0.01;
String stringValue = "";

void setup() {
```

```

Serial.begin(115200);

// Connect to Wi-Fi
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to Wi-Fi");
while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(300);
}
Serial.println();

// Configure SSL client
ssl_client.setInsecure();
ssl_client.setConnectionTimeout(1000);
ssl_client.setHandshakeTimeout(5);

// Initialize Firebase
initializeApp(aClient, app, getAuth(user_auth), processData, "🔒 authTask");
app.getApp<RealtimeDatabase>(Database);
Database.url(DATABASE_URL);
}

void loop(){
    // Maintain authentication and async tasks
    app.loop();
    // Check if authentication is ready
    if (app.ready()){
        // Periodic data sending every 10 seconds
        unsigned long currentTime = millis();
        if (currentTime - lastSendTime >= sendInterval){
            // Update the last send time
            lastSendTime = currentTime;

            // send a string
            stringValue = "value_" + String(currentTime);
            Database.set<String>(aClient, "/test/string", stringValue, processData,
"RTDB_Send_String");
            // send an int
            Database.set<int>(aClient, "/test/int", intValue, processData, "RTDB_Send_Int");
            intValue++; //increment intValue in every loop

            // send a string
            floatValue = 0.01 + random (0,100);
            Database.set<float>(aClient, "/test/float", floatValue, processData,
"RTDB_Send_Float");
        }
    }
}

void processData(AsyncResult &aResult) {
    if (!aResult.isResult())
        return;
}

```



```

if (aResult.isEvent())
    Firebase.printf("Event task: %s, msg: %s, code: %d\n", aResult.uid().c_str(),
aResult.eventLog().message().c_str(), aResult.eventLog().code());

if (aResult.isDebug())
    Firebase.printf("Debug task: %s, msg: %s\n", aResult.uid().c_str(),
aResult.debug().c_str());

if (aResult.isError())
    Firebase.printf("Error task: %s, msg: %s, code: %d\n", aResult.uid().c_str(),
aResult.error().message().c_str(), aResult.error().code());

if (aResult.available())
    Firebase.printf("task: %s, payload: %s\n", aResult.uid().c_str(), aResult.c_str());
}

```

Debes insertar tus credenciales de red, URL de la base de datos, clave API del proyecto, correo electrónico y contraseña del usuario de Firebase.

Sube el código a tu placa ESP32. No olvides insertar tus credenciales de red, la URL de la base de datos, la clave API del proyecto y el correo electrónico y la contraseña del usuario del proyecto de Firebase.

Después de cargar el código, abra el Monitor Serial a una velocidad en baudios de 115200 y presione el botón de reinicio integrado del ESP32 para que comience a ejecutar el código.

Si todo funciona como se espera, los valores deberían almacenarse en la base de datos y deberías recibir mensajes de éxito, *Ilustración 2.23*.

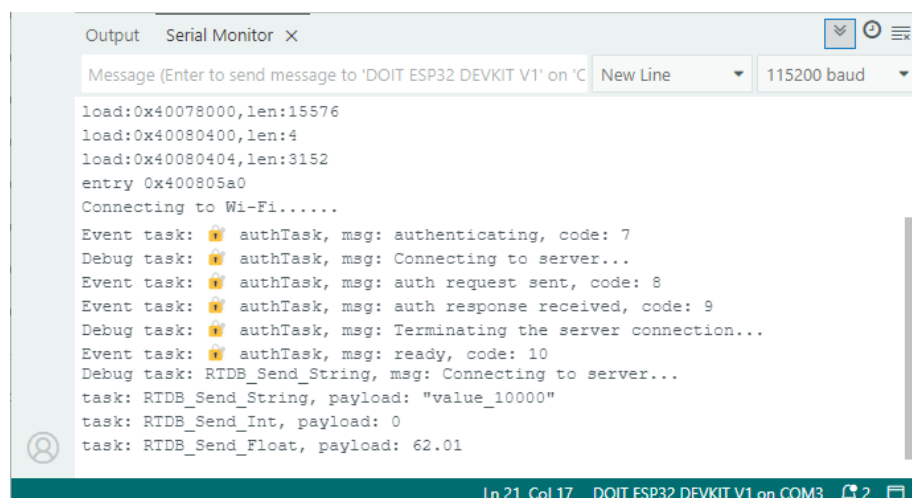


Ilustración 2.23 – Envío de datos exitoso a Firebase

Accede a la base de datos Firebase Realtime de tu proyecto y verás los valores guardados en las diferentes rutas de nodo. Cada 10 segundos, se guarda un nuevo valor. La base de datos parpadea cuando se guardan nuevos valores, *Ilustración 2.24*.



Ilustración 2.24 – La página de la base de datos

2.7 Lectura desde Firebase RTDB (modo asíncrono)

El siguiente programa lee los valores almacenados en la base de datos mediante un método asíncrono. Suba el siguiente código a su placa. Puede usar la misma placa ESP32 u otra para obtener los datos publicados por el ESP32 anterior.

```
/*  
  Rui Santos & Sara Santos - Random Nerd Tutorials  
  Complete instructions at https://RandomNerdTutorials.com/esp32-firebase-realtime-database/  
  */  
  
#include <Arduino.h>  
#include <WiFi.h>  
#include <WiFiClientSecure.h>  
#include <FirebaseClient.h>  
  
// Network and Firebase credentials  
#define WIFI_SSID "REPLACE_WITH_YOUR_SSID"  
#define WIFI_PASSWORD "REPLACE_WITH_YOUR_PASSWORD"  
  
#define Web_API_KEY "REPLACE_WITH_YOUR_FIREBASE_PROJECT_API_KEY"  
#define DATABASE_URL "REPLACE_WITH_YOUR_FIREBASE_DATABASE_URL"  
#define USER_EMAIL "REPLACE_WITH_FIREBASE_PROJECT_EMAIL_USER"  
#define USER_PASS "REPLACE_WITH_FIREBASE_PROJECT_USER_PASS"  
  
// User function  
void processData(AsyncResult &aResult);
```

```

// Authentication
UserAuth user_auth(Web_API_KEY, USER_EMAIL, USER_PASS);

// Firebase components
FirebaseApp app;
WiFiClientSecure ssl_client;
using AsyncClient = AsyncClientClass;
AsyncClient aClient(ssl_client);
RealtimeDatabase Database;

// Timer variables for reading data every 10 seconds
unsigned long lastSendTime = 0;
const unsigned long sendInterval = 10000; // 10 seconds in milliseconds

// Variables to save values from the database
int intValue;
float floatValue;
String stringValue;

void setup(){
    Serial.begin(115200);

    // Connect to Wi-Fi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(300);
    }
    Serial.println();
    Serial.print("Connected with IP: ");
    Serial.println(WiFi.localIP());
    Serial.println();

    // Configure SSL client
    ssl_client.setInsecure();
    ssl_client.setConnectionTimeout(1000);
    ssl_client.setHandshakeTimeout(5);

    // Initialize Firebase
    initializeApp(aClient, app, getAuth(user_auth), processData, "🔒 authTask");
    app.getApp<RealtimeDatabase>(Database);
    Database.url(DATABASE_URL);
}

void loop(){
    // Maintain authentication and async tasks
    app.loop();

    // Check if authentication is ready
    if (app.ready()){
        // Getting data every 10 seconds
        unsigned long currentTime = millis();

```

```

    if (currentTime - lastSendTime >= sendInterval){
        // Update the last send time
        lastSendTime = currentTime;

        // GET VALUES FROM DATABASE (using the callback async method method)
        // you can then get the values on the processData function as soon as the results
are available
        Database.get(aClient, "/test/int", processData, false, "RTDB_GetInt");
        Database.get(aClient, "/test/float", processData, false, "RTDB_GetFloat");
        Database.get(aClient, "/test/string", processData, false, "RTDB_GetString");

        Serial.println("Requested data from /test/int, /test/float, and /test/string");
    }
}

void processData(AsyncResult &aResult){
    if (!aResult.isResult())
        return;

    if (aResult.isEvent())
        Firebase.printf("Event task: %s, msg: %s, code: %d\n", aResult.uid().c_str(),
aResult.eventLog().message().c_str(), aResult.eventLog().code());

    if (aResult.isDebug())
        Firebase.printf("Debug task: %s, msg: %s\n", aResult.uid().c_str(),
aResult.debug().c_str());

    if (aResult.isError())
        Firebase.printf("Error task: %s, msg: %s, code: %d\n", aResult.uid().c_str(),
aResult.error().message().c_str(), aResult.error().code());

    // here you get the values from the database and save them in variables if you need to
use them later
    if (aResult.available()) {
        // Log the task and payload
        Firebase.printf("task: %s, payload: %s\n", aResult.uid().c_str(), aResult.c_str());

        // Extract the payload as a String
        String payload = aResult.c_str();

        /// Handle int from /test/int
        if (aResult.uid() == "RTDB_GetInt"){
            // Extract the value as an int
            intValue = payload.toInt();
            Firebase.printf("Stored intValue: %d\n", intValue);
        }
        // Handle float from /test/float
        else if (aResult.uid() == "RTDB_GetFloat"){
            // Extract the value as a float
            floatValue = payload.toFloat();
            Firebase.printf("Stored floatValue: %.2f\n", floatValue);
        }
    }
}

```

```

// Handle String from /test/string
else if (aResult.uid() == "RTDB_GetString"){
    // Extract the value as a String
    stringValue = payload;
    Firebase.printf("Stored stringValue: %s\n", stringValue.c_str());
}
}
}

```

Debe insertar sus credenciales de red, URL de la base de datos, clave API del proyecto, correo electrónico y contraseña del usuario de Firebase.

Sube el código a tu placa. A continuación, abre el Monitor Serie a una velocidad de 115200 baudios. Tras unos segundos, se imprimirán los valores guardados en la base de datos, *Ilustración 2.25*.

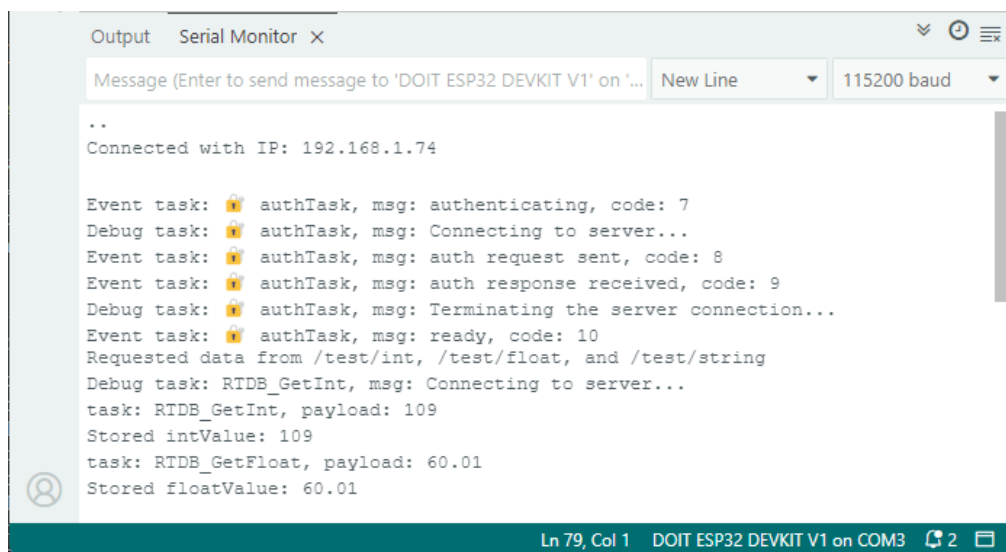


Ilustración 2.25 – Almacenamiento de los valores

2.8 Lectura desde Firebase RTDB (sin modo asíncrono)

Para aplicaciones más simples donde no necesita poner en cola tareas y el código de bloqueo no es crítico mientras espera una operación de base de datos, puede usar el `get()` función en un formato simplificado. Esto permite recuperar directamente un valor de Firebase Realtime Database y almacenarlo inmediatamente en una variable.

Por ejemplo:

```
int intValue = Database.get<int>(aClient, "/test/int");
```

El `Database.get(aClient, "/test/int")` obtiene sincrónicamente un número entero de la ruta RTDB de Firebase `/test/int` usando `aClient`, lo asigna a `intValue` y bloquea la ejecución hasta que se complete. Esto es ideal para aplicaciones sencillas donde no se necesitan múltiples tareas, pero puede retrasar otras operaciones.

Aquí tienes un código que utiliza este método. Funciona exactamente igual que el ejemplo anterior. Sin embargo, para aplicaciones más complejas, deberías optar por el método asíncrono.

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <FirebaseClient.h>

// Network and Firebase credentials
#define WIFI_SSID "REPLACE_WITH_YOUR_SSID"
#define WIFI_PASSWORD "REPLACE_WITH_YOUR_PASSWORD"

#define Web_API_KEY "REPLACE_WITH_YOUR_FIREBASE_PROJECT_API_KEY"
#define DATABASE_URL "REPLACE_WITH_YOUR_FIREBASE_DATABASE_URL"
#define USER_EMAIL "REPLACE_WITH_FIREBASE_PROJECT_EMAIL_USER"
#define USER_PASS "REPLACE_WITH_FIREBASE_PROJECT_USER_PASS"

// User function
void processData(AsyncResult &aResult);

// Authentication
UserAuth user_auth(Web_API_KEY, USER_EMAIL, USER_PASS);

// Firebase components
FirebaseApp app;
WiFiClientSecure ssl_client;
using AsyncClient = AsyncClientClass;
AsyncClient aClient(ssl_client);
RealtimeDatabase Database;

// Timer variables for sending data every 10 seconds
unsigned long lastSendTime = 0;
const unsigned long sendInterval = 10000; // 10 seconds in milliseconds

// Variables to save values from the database
int intValue;
float floatValue;
String stringValue;

void setup() {
    Serial.begin(115200);

    // Connect to Wi-Fi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(300);
    }
    Serial.println();
    Serial.print("Connected with IP: ");
    Serial.println(WiFi.localIP());
```

```

Serial.println();

// Configure SSL client
ssl_client.setInsecure();
ssl_client.setConnectionTimeout(1000);
ssl_client.setHandshakeTimeout(5);

// Initialize Firebase
initializeApp(aClient, app, getAuth(user_auth), processData, "🔒 authTask");
app.getApp<RealtimeDatabase>(Database);
Database.url(DATABASE_URL);
}

void loop(){
    // Maintain authentication and async tasks
    app.loop();

    // Check if authentication is ready
    if (app.ready()){
        // Periodic data sending every 10 seconds
        unsigned long currentTime = millis();
        if (currentTime - lastSendTime >= sendInterval){
            // Update the last send time
            lastSendTime = currentTime;

            // GET VALUES FROM DATABASE (and save the data in a variable)
            int intValue = Database.get<int>(aClient, "/test/int");
            check_and_print_value (intValue);

            float floatValue = Database.get<float>(aClient, "/test/float");
            check_and_print_value(floatValue);

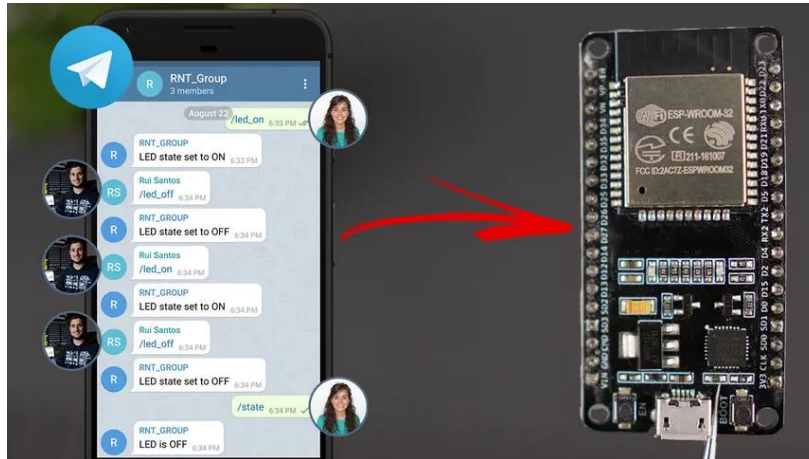
            String stringValue = Database.get<String>(aClient, "/test/string");
            check_and_print_value(stringValue);

            Serial.println("Requested data from /test/int, /test/float, and /test/string");
        }
    }
}

template <typename T>
void check_and_print_value(T value){
    // To make sure that we actually get the result or error.
    if (aClient.lastError().code() == 0){
        Serial.print("Success, Value: ");
        Serial.println(value);
    }
    else {
        Firebase.printf("Error, msg: %s, code: %d\n", aClient.lastError().message().c_str(),
aClient.lastError().code());
    }
}

```


PRÁCTICA III – TELEGRAM Y ESP32



OBJETIVO DE LA UNIDAD:

Unidad IV: El alumno desarrollará un sistema de IoT para monitoreo de variables físicas de un proceso con almacenamiento e interacción en la nube.

OBJETIVOS DE LA PRACTICA:

Monitorear y controlar variables de manera remota usando la red social de Telegram.

ANTECEDENTES

La integración entre un ESP32 y un bot de Telegram, (*Ilustración 3.1*), representa una solución práctica y accesible para implementar sistemas de monitoreo y control a distancia. Actividades como el encendido y apagado de LEDs o la consulta de datos ambientales, como la temperatura, se pueden realizar mediante comandos desde cualquier lugar con acceso a internet.



Ilustración 3.1 – Telegram Bot

Los usos que se le pueden dar a esta plataforma son:

1. Automatización y control remoto

Mediante la creación de bots personalizados, Telegram permite automatizar tareas o controlar dispositivos a distancia. Esto es especialmente útil en proyectos de IoT, donde un microcontrolador como el ESP32 puede interactuar con el bot para ejecutar acciones como encender luces, leer sensores o activar sistemas de seguridad.

2. Notificaciones en tiempo real Telegram se puede utilizar para enviar alertas automáticas desde sistemas integrados, por ejemplo: advertencias por alta temperatura, detección de movimiento, fallas en un sistema, entre otros.

3. Gestión de sistemas domésticos o industriales

Es posible implementar sistemas de domótica, donde el usuario puede controlar el estado de luces, ventiladores, cámaras o cerraduras desde cualquier lugar mediante comandos simples enviados al bot.

4. Monitoreo de variables

El bot puede solicitar información como la temperatura, humedad o nivel de un sensor conectado al ESP32, y el sistema responderá con los datos actuales, funcionando como una interfaz de monitoreo remota.

5. Interacción personalizada

Se pueden programar menús, botones o respuestas automáticas, lo que hace que Telegram sea una alternativa amigable como interfaz de usuario para aplicaciones tecnológicas sin necesidad de desarrollar una app desde cero.

6. Seguridad y privacidad

Telegram permite restringir el acceso al bot a usuarios específicos mediante IDs, lo cual mejora la seguridad del sistema al evitar que personas no autorizadas lo controlen.

MATERIAL POR UTILIZAR

- ESP32 con modulo
- Dos leds (verde y rojo)
- Jumpers Macho-Macho
- Sensor de temperatura (DHT11)
- Protoboard
- Dos resistencias de 330Ω

SOFTWARE NECESARIO

Telegram: <https://web.telegram.org/k/>

Arduino IDE: <https://www.arduino.cc/en/software/>

DESARROLLO DE LA PRÁCTICA

Práctica 3. Encendido de leds y monitoreo de temperatura con bot de telegram.

3.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 3.2*

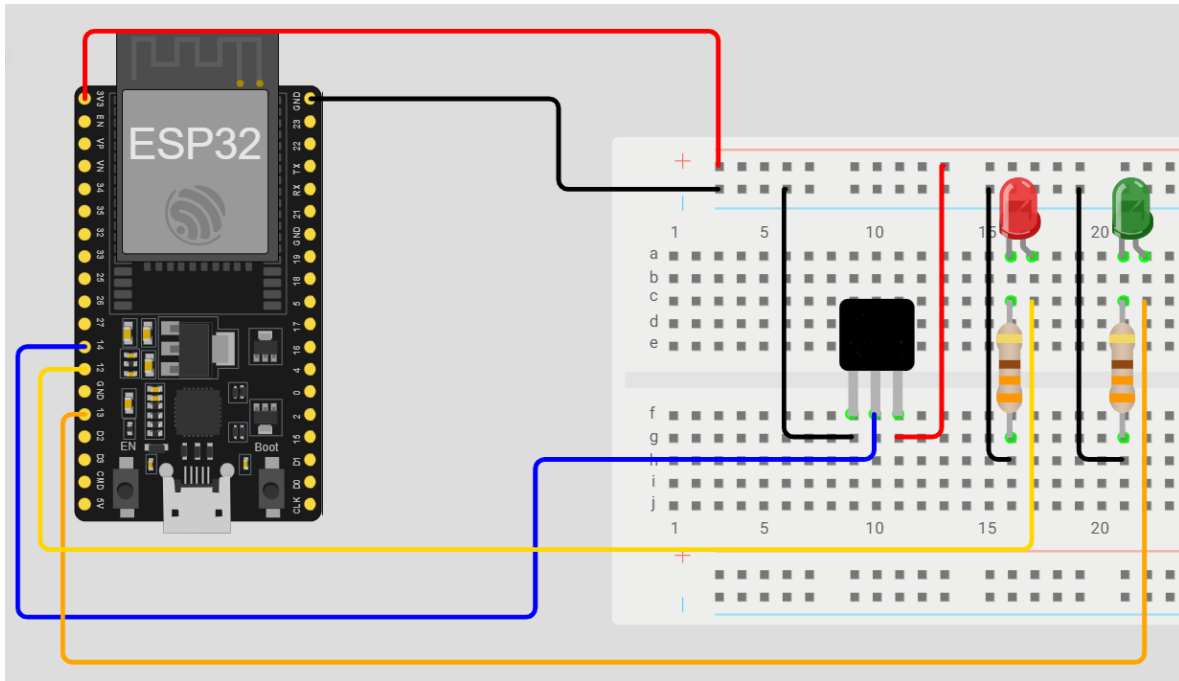


Ilustración 3.2 – Diagrama a implementar

3.2 Configuración y programación de la plataforma Telegram

Para iniciar se tiene que buscar un Bot oficial de Telegram para crear un Bot, *Ilustración 3.3*.

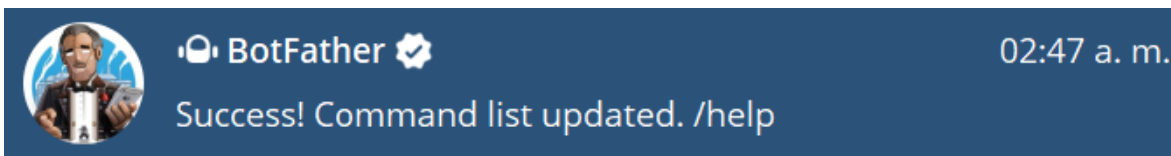


Ilustración 3.3 – Bot Oficial

Como siguiente paso se realiza la creación del bot, con el comando `/newbot` y le asignamos un nombre. También hay que crear un nombre de usuario, ya que el usuario será con el que se identifique al bot en el apartado de chats.

Al final se obtiene un API o token para el bot personalizado que se utilizara en el código de Arduino. Este Token hay que copiarlo, ya que servirá en el código de Arduino, *Ilustración 3.4*.

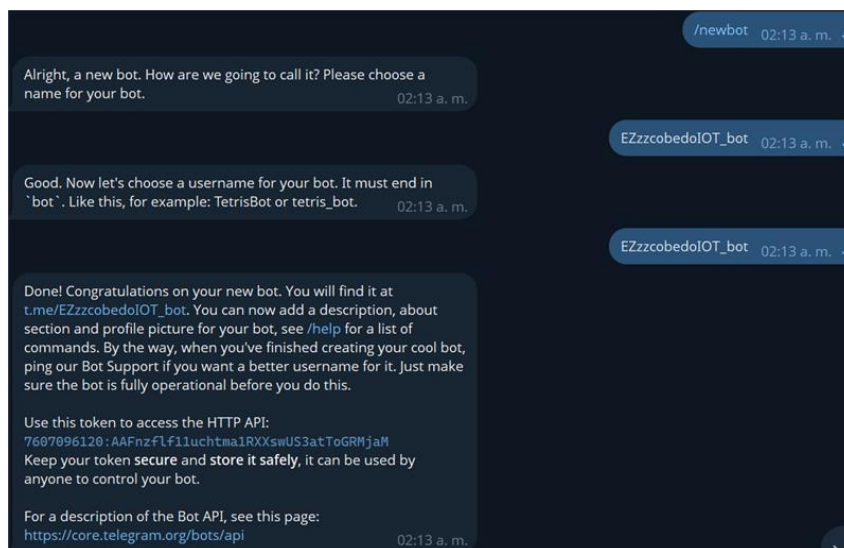


Ilustración 3.4 – Apartado de creación de bot

En este paso se colocan los comandos `/setcommands`, luego se selecciona el bot para los comandos y se crea la lista y la descripción de cada uno de los comandos como se muestra en la *ilustración 3.5*.

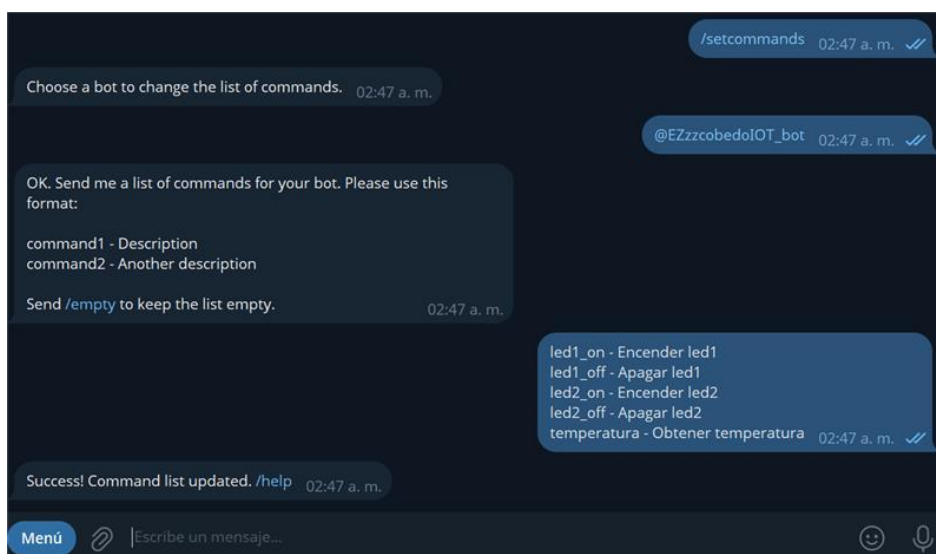


Ilustración 3.5 – Agregar lista de comandos

Se ocuparon las siguientes librerías que se muestran en la *Ilustración 3.6*.

```
#include <WiFi.h>
#include <UniversalTelegramBot.h>
#include <WiFiClientSecure.h>
#include <DHT.h>
```

Ilustración 3.6 – Librerías necesarias

Para configurar la red WiFi se agrega: el nombre de la red, la contraseña y el Token proporcionado por Telegram anteriormente, *Ilustración 3.7*.

```
const char* ssid      = "Alumnos Telemática";
const char* password  = "telematica";
const char* telegramToken = "7607096120:AAFnzflf11uchtma1RXXswUS3atToGRMjaM";
```

Ilustración 3.7 – Configuración de red WIFI

Ahora se asigna los pines GPIO de los leds y del sensor, *Ilustración 3.8*.

```
const int ledPin1 = 12;
const int ledPin2 = 13;
#define DHTPIN 14
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

Ilustración 3.8 – Definición de variables

Se realiza la conexión del ESP32 a una red WiFi y al bot de Telegram. Primero, se inicializa el sensor DHT con **dht.begin()** para poder obtener lecturas de temperatura más adelante. Luego, se comienza la conexión WiFi utilizando el nombre y la contraseña de la red con **WiFi.begin()**, y mediante un bucle **while** se espera la conexión. Una vez establecida la conexión, se imprime un mensaje junto con la dirección IP local asignada al dispositivo, se utiliza **client.setInsecure()** para permitir conexiones HTTPS sin verificación de certificados, algo necesario para comunicarse con Telegram.

Finalmente, con **bot.getMe()** se verifica la conexión al bot de Telegram, obteniendo información básica de este y asegurando que la comunicación está funcionando correctamente, *Ilustración 3.9*.

```
dht.begin();

Serial.print("Conectando a WiFi");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
}
Serial.println("\nConexión WiFi establecida");
Serial.print("IP: ");
Serial.println(WiFi.localIP());

client.setInsecure();
Serial.println("Conectando al bot de Telegram...");
Serial.println(bot.getMe());
}
```

Ilustración 3.9 – conexión a la red y enlace al bot

En esta sección del código, el ESP32 revisa si el bot de Telegram ha recibido nuevos mensajes. Para cada mensaje nuevo, se extrae el texto y el identificador del chat. Luego, se analiza el contenido del mensaje y, dependiendo del comando recibido (`/led1_on`, `/led1_off`, `/led2_on` o `/led2_off`), se enciende o apaga el LED correspondiente utilizando **digitalWrite**. Después de ejecutar la acción, el bot responde al usuario confirmando el estado del LED mediante un mensaje de texto, *Ilustración 3.10*.

```
int numNew = bot.getUpdates(bot.last_message_received + 1);
for (int i = 0; i < numNew; i++) {
    String chat_id = bot.messages[i].chat_id;
    String text = bot.messages[i].text;
    Serial.print("Mensaje: ");
    Serial.println(text);
    if (text == "/led1_on") {
        digitalWrite(ledPin1, HIGH);
        bot.sendMessage(chat_id, "LED 1 encendido", "");
    }
    else if (text == "/led1_off") {
        digitalWrite(ledPin1, LOW);
        bot.sendMessage(chat_id, "LED 1 apagado", "");
    }
    else if (text == "/led2_on") {
        digitalWrite(ledPin2, HIGH);
        bot.sendMessage(chat_id, "LED 2 encendido", "");
    }
    else if (text == "/led2_off") {
        digitalWrite(ledPin2, LOW);
        bot.sendMessage(chat_id, "LED 2 apagado", "");
    }
}
```

Ilustración 3.10 – comandos para los leds.

El ESP32 verifica si el mensaje recibido es el comando `/temperatura`. Si lo es, se realizan lecturas de temperatura y humedad mediante el sensor DHT. Si los datos obtenidos no son válidos (por ejemplo, si el sensor está desconectado o falla), se envía un mensaje de error al usuario por medio del bot de Telegram, (*Ilustración 3.11*).

```
else if (text == "/temperatura") {
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) {
        bot.sendMessage(chat_id, "Error al leer sensor DHT", "");
        Serial.println("Error DHT");
    } else {
        char msg[64];
        snprintf(msg, sizeof(msg),
            "🌡 Temperatura: %.1f °C\n💧 Humedad: %.1f %%", t, h);
        bot.sendMessage(chat_id, msg, "");
        Serial.println(msg);
    }
} else {
    bot.sendMessage(chat_id,
        "Comando no reconocido.\nUsa /led1_on, /led1_off, /led2_on, /led2_off o /temperatura",
        "");
}
//
delay(1000);
```

Ilustración 3.11 – código para sensor y sugerencias de entrada

Se envía como respuesta al usuario, mostrando los valores actuales del ambiente. Además, si el comando recibido no coincide con ninguno de los que el código reconoce (como `/led1_on`, `/led1_off`, `/led2_on`, `/led2_off` o `/temperatura`), se responde automáticamente con un mensaje que indica que el comando no es reconocido, junto con la lista de comandos válidos.

Cabe destacar que los mensajes que se interpretan como comandos están definidos dentro de las condiciones `if (text == "/comando")` y pueden ser modificados o ampliados directamente en esta parte del código. Por ejemplo, si se desea agregar un nuevo comando, basta con añadir una nueva condición `else if (text == "/nuevo_comando")` y definir qué acción debe ejecutarse cuando se reciba dicho texto.

En la *Ilustración 3.12* se muestra el Bot de telegram en uso y el resultado de la práctica.



Ilustración 3.12 – Resultado

En la *Ilustración 3.13* se muestran algunas sugerencias de comandos a utilizar.

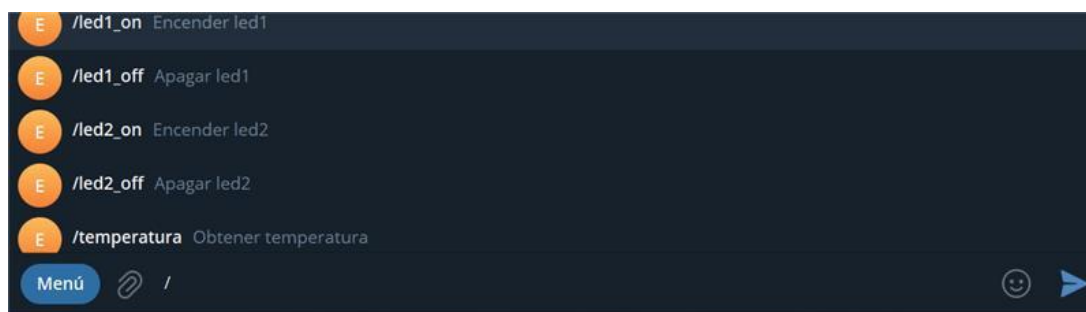


Ilustración 3.13 – Sugerencias

A continuación, se muestra el código completo para su implementación:

```
#include <WiFi.h>
#include <UniversalTelegramBot.h>
#include <WiFiClientSecure.h>
#include <DHT.h>
const char* ssid      = "Alumnos Telemática";
const char* password = "telematica";
const char* telegramToken = "7607096120:AAFnzflf11uchtma1RXXswUS3atToGRMjaM";

WiFiClientSecure client;
UniversalTelegramBot bot(telegramToken, client);

const int ledPin1 = 12;
const int ledPin2 = 13;
#define DHTPIN 14
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
    // Monitor serie
    Serial.begin(115200);
    delay(1000);

    // Pines como salidas
    pinMode(ledPin1, OUTPUT);
    pinMode(ledPin2, OUTPUT);

    dht.begin();

    Serial.print("Conectando a WiFi");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nConexión WiFi establecida");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());

    client.setInsecure();
    Serial.println("Conectando al bot de Telegram...");
    Serial.println(bot.getMe());
}

void loop() {
```

```

// Reintentar WiFi si se cae
if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi desconectado. Reconectando...");
    WiFi.begin(ssid, password);
    delay(5000);
    return;
}

int numNew = bot.getUpdates(bot.last_message_received + 1);
for (int i = 0; i < numNew; i++) {
    String chat_id = bot.messages[i].chat_id;
    String text    = bot.messages[i].text;
    Serial.print("Mensaje: ");
    Serial.println(text);
    if (text == "/led1_on") {
        digitalWrite(ledPin1, HIGH);
        bot.sendMessage(chat_id, "LED 1 encendido", "");
    }
    else if (text == "/led1_off") {
        digitalWrite(ledPin1, LOW);
        bot.sendMessage(chat_id, "LED 1 apagado", "");
    }
    else if (text == "/led2_on") {
        digitalWrite(ledPin2, HIGH);
        bot.sendMessage(chat_id, "LED 2 encendido", "");
    }
    else if (text == "/led2_off") {
        digitalWrite(ledPin2, LOW);
        bot.sendMessage(chat_id, "LED 2 apagado", "");
    }
    else if (text == "/temperatura") {
        float h = dht.readHumidity();
        float t = dht.readTemperature();
        if (isnan(h) || isnan(t)) {
            bot.sendMessage(chat_id, "Error al leer sensor DHT", "");
            Serial.println("Error DHT");
        } else {
            char msg[64];
            snprintf(msg, sizeof(msg),
                "🌡 Temperatura: %.1f °C\n💧 Humedad: %.1f %%", t, h);
            bot.sendMessage(chat_id, msg, "");
            Serial.println(msg);
        } }
    else {
        bot.sendMessage(chat_id,

```

```
"/temperatura",
```

```
"" );
```

```
delay(1000);
```

Conclusiones

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There is no handwriting or other markings on the paper.

PRÁCTICA IV - CONEXIÓN A WHATSAPP



Objetivo de la unidad:

Unidad IV: El alumno desarrollará un sistema de IoT para monitoreo de variables físicas de un proceso con almacenamiento e interacción en la nube.

Objetivos de la práctica:

Implementar circuitos que puedan enviar y recibir datos con otros dispositivos a través de la red social de WhatsApp.

Antecedentes

ThingESP

La biblioteca y plataforma ThingESP, (*Ilustración 4.1*) de Arduino es una biblioteca de cliente fácil de usar para conectar sus dispositivos IoT a la plataforma en la nube ThingESP. Esta es una biblioteca diseñada específicamente para Arduino IDE, por lo que puede instalarla fácilmente en su entorno y comenzar a conectar sus dispositivos en cuestión de minutos. Admite múltiples interfaces de red como Wifi.

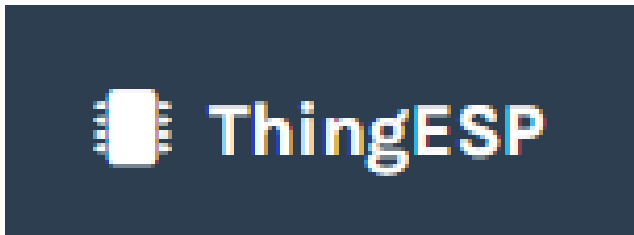


Ilustración 4.1 - Plataforma ThingESP

Twilio

Esta es una de las principales plataformas de comunicación en la nube que permite atraer clientes a través de los distintos canales: SMS, voz, video, correo electrónico, WhatsApp y más, *Ilustración 4.2*.



Ilustración 4.2 - Plataforma Twilio

Material por utilizar

- Dispositivo móvil con acceso a internet
- Protoboard
- Tarjeta Nodemcu ESP-32
- Led de rojo
- Resistencia de 330Ω
- Resistencia de $4.7k\Omega$
- Sensor de temperatura DS18B20
- Cables dupont

Software Necesario

- IDE de Arduino: <https://www.arduino.cc/en/software>
- Librería y aplicación ThingESP: <https://thingesp.siddhesh.me>
- Plataforma Twilio: <https://www.twilio.com/es-mx>
- Whatsapp

Práctica 4.1 Activar cargas a través de mensajes de WhatsApp

4.1.1 Implementa el circuito del diagrama que se muestra en la *Ilustración 4.3*.

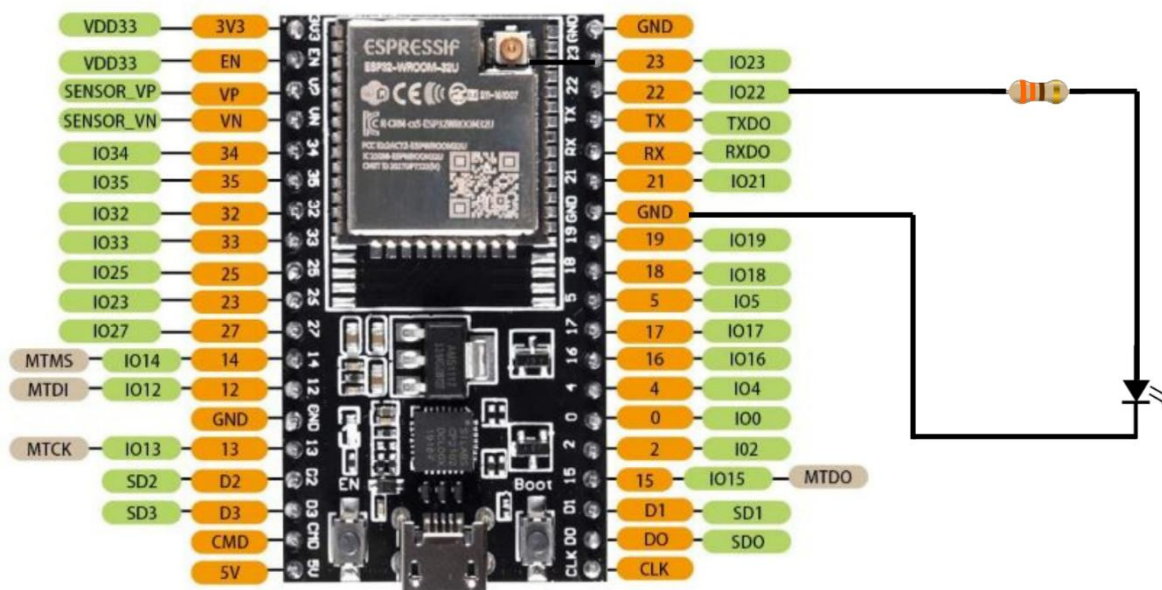


Ilustración 4.3 - Diagrama de conexión para el LED

El circuito implementado se muestra en *la Ilustración 4.4*.

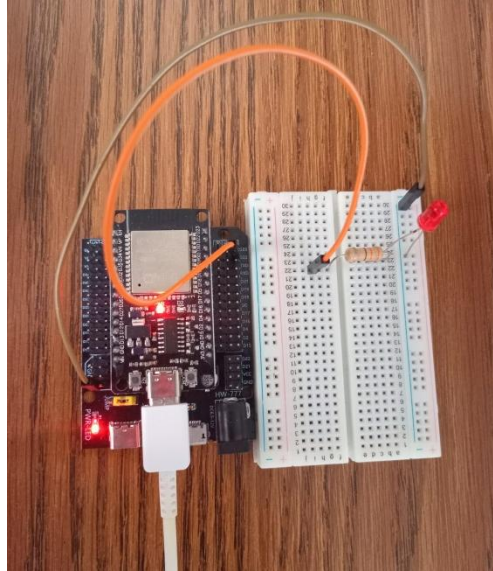


Ilustración 4.4 - Implementación del diagrama de conexión

4.1.2 Captura el siguiente código en el IDE de Arduino.

```
#include <WiFi.h>
#include <ThingESP.h>
```

```
ThingESP32 thing("ayaair0205", "LedsWhatsApp", "ledonoff");
constexpr uint8_t LED = 22;
```

```
void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
  Serial.println("Conectando al WiFi...");
  thing.SetWiFi("INFINITUM3C37", "75mz4rUODt");
  thing.initDevice();          // Intenta registrar el dispositivo
  Serial.println("Esperando comandos de WhatsApp");
}
```

```
String HandleResponse(String query) {
  query.toLowerCase();        // Acepta "Led ON", "LED on", etc.
  if (query == "led on") { digitalWrite(LED, HIGH); return "Done: LED Turned ON"; }
  if (query == "led off") { digitalWrite(LED, LOW); return "Done: LED Turned OFF"; }
  if (query == "led status")
    return digitalRead(LED) ? "LED is ON" : "LED is OFF";
  return "Your query was invalid.";
}
```

```
void loop() { thing.Handle(); }
```

NOTA: En las siguientes líneas de código tendremos que realizar algunas modificaciones, estas se explicaran más adelante.

- ThingESP32 thing("ayaair0205", "LedsWhatsApp", "ledonoff");
- thing.SetWiFi("INFINITUM3C37", "75mz4rUODt");

4.1.3 Configuración de las aplicaciones para la conexión con la tarjeta ESP32.

Como primer paso, debemos instalar la librería “ThingESP”, para esto nos dirigimos al “Library Manager” y buscamos la librería mencionada y se procede a instalarla. Como se observa en la *Ilustración 4.5*.

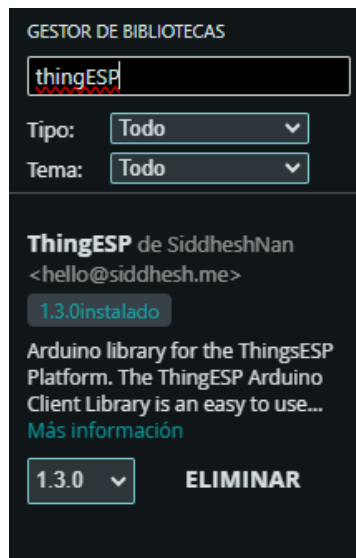


Ilustración 4.5 - Instalación de la librería ThingESP

Lo siguiente es crear una cuenta en la plataforma ThingESP. Para eso, utilizaremos el siguiente enlace: <https://thingsp.siddhesh.me>

Ya en la página, nos iremos al botón que dice “Crear Cuenta”, *Ilustración 4.6*.

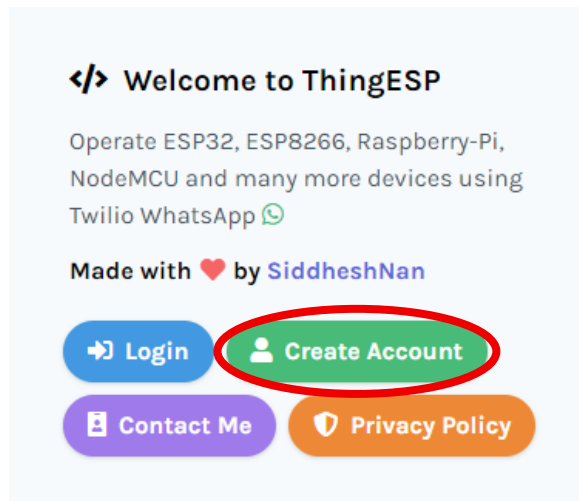


Ilustración 4.6 - Creación de nuestra cuenta en ThingESP

Ingresaremos nuestros datos, marcaremos la casilla “No soy un robot” y finalmente nos registraremos en el botón “Registrar”, *Ilustración 4.7*.

Ilustración 4.7 - Registro de nuestros datos

Para concluir con el proceso de registro de la cuenta tendremos que confirmar nuestra dirección de correo, por lo que te aparecerá el mensaje de la *Ilustración 4.8*.

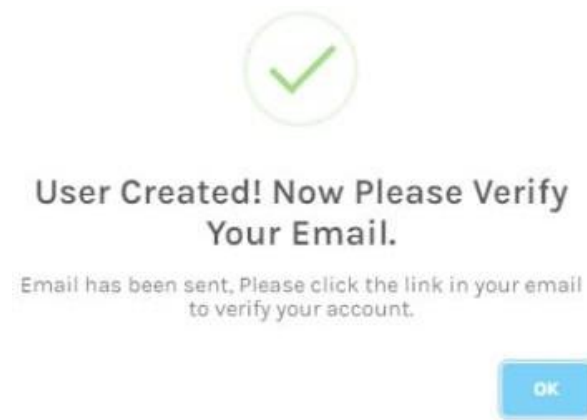


Ilustración 4.8 - Verificación necesaria

Procedemos con la confirmación de nuestro correo electrónico, *Ilustración 4.9*.

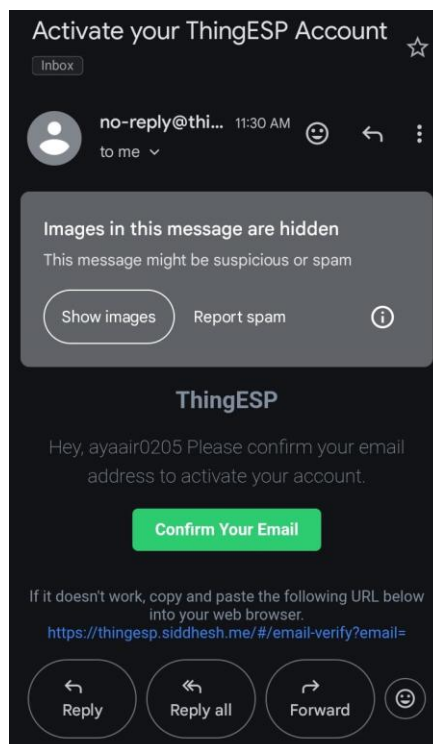
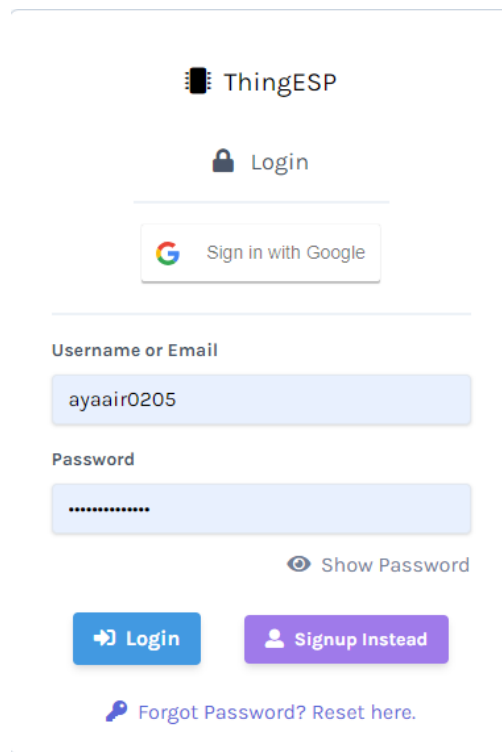


Ilustración 4.9 - Confirmación de nuestro correo

Ahora, ya podemos iniciar sesión con nuestro usuario y contraseña, *Ilustración 4.10*.



The login form for ThingESP is centered on the page. At the top is the ThingESP logo. Below it is a 'Login' button with a lock icon. A 'Sign in with Google' button is positioned below the login button. The form contains two input fields: 'Username or Email' with the value 'ayaair0205' and 'Password' with masked characters. A 'Show Password' link is next to the password field. At the bottom are 'Login' and 'Signup Instead' buttons, and a 'Forgot Password? Reset here.' link.

Ilustración 4.10 - Inicio de sesión con nuestros datos

Al iniciar sesión nos aparecerá la siguiente página, *Ilustración 4.11*.

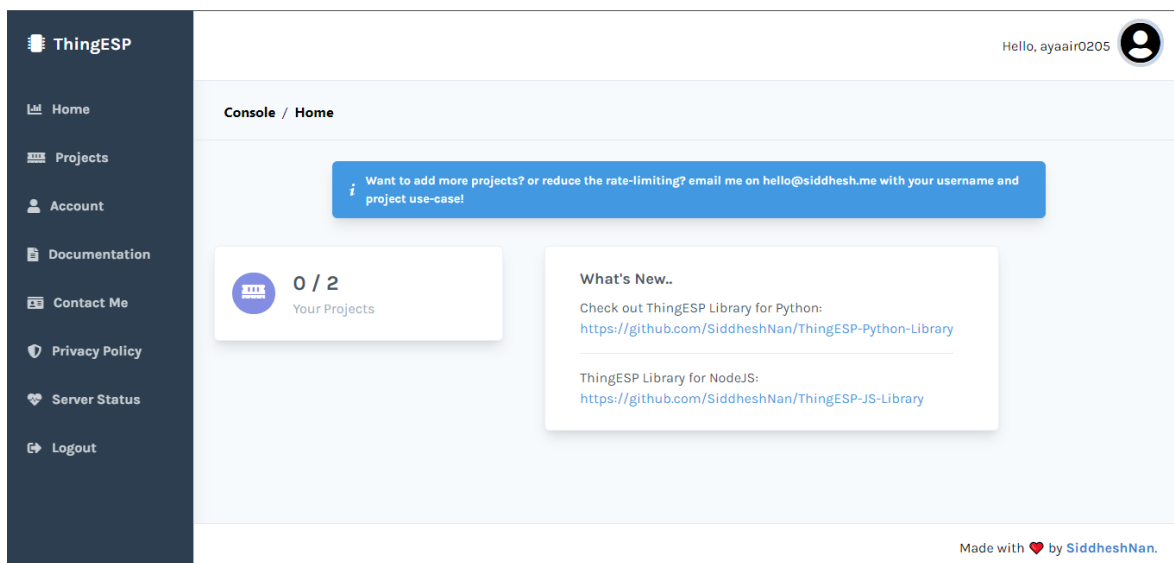


Ilustración 4.11 - Página de inicio de ThingESP

Como siguiente paso, procederemos a crear nuestro proyecto, para eso nos iremos a la pestaña “Proyectos” y seguidamente presionaremos el botón “Add a new project”. Esto se muestra en la *Ilustración 4.12*.

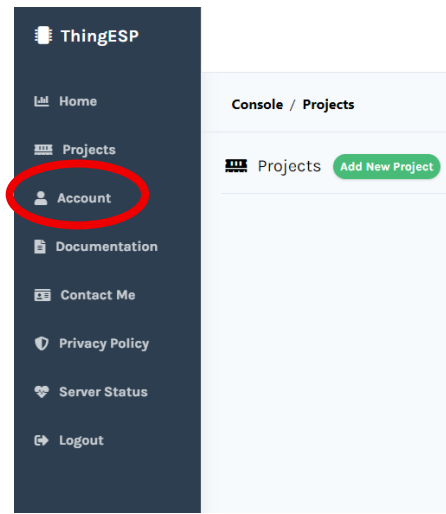


Ilustración 4.12 - Pestaña de creación del proyecto

Al presionar “Add a new Project” nos saldrá la siguiente ventana, *Ilustración 4.13*.

+ Create New Project (2 remaining out of 2)

Project Name (without spaces or special characters)

LedsWhatsApp

Project's Device Credentials

ledonoff

Device Calls: Send Messages From Device (Requires Twilio SID and AuthToken)

☐

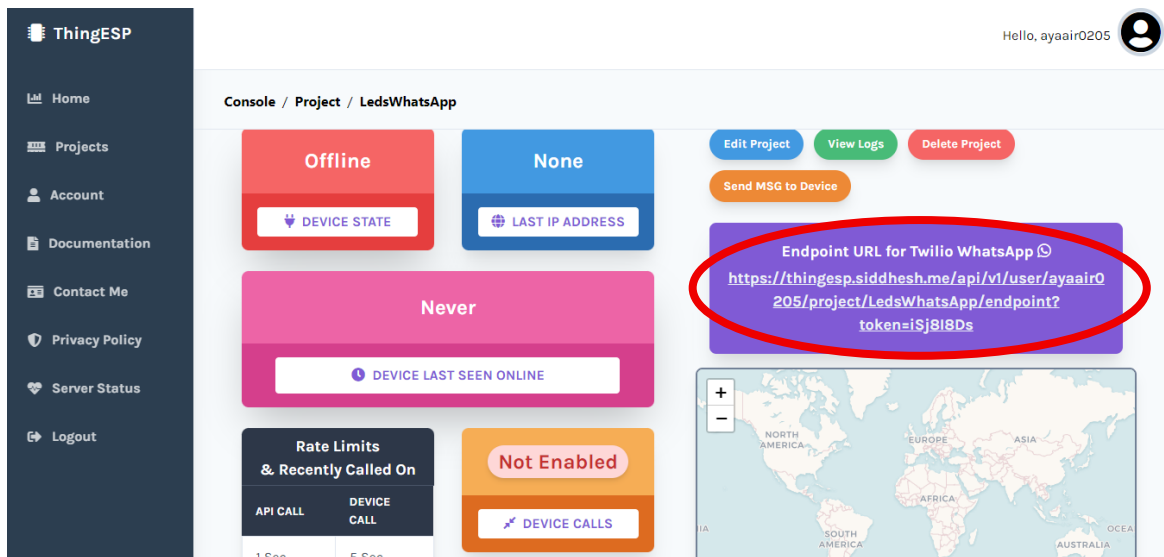
Submit **Go Back**

Ilustración 4.13 - Creación del nombre y credenciales del Proyecto

En esta ventana le pondremos un nombre al proyecto al igual que nombraremos nuestras credenciales, estos podrán variar, pero son importantes a la hora de realizar nuestro código, por lo tanto, hay que escoger nombres sencillos para no complicar o cometer errores de gramática. Al elegir los nombres de ambos campos daremos click en “Submit”.

Al crear nuestro proyecto, nos aparecerá la ventana de la *Ilustración 4.14*.

En esta ventana, nos aparecerá un enlace en el apartado “Endpoint URL for Twilio Whatsapp”, debemos guardar este enlace ya que nos será de utilidad al utilizar la aplicación Twilio, la cual se explicará más adelante.



Ilustración

4.14 - Proyecto creado

Por el momento, se ha concluido con la configuración de ThingESP. Cabe recalcar que es importante implementar las siguientes librerías en nuestro código, *Ilustración 4.15*.

```
#include <WiFi.h>
#include <ThingESP.h>

ThingESP32 thing("ayaair0205", "LedsWhatsApp", "ledonoff");
```

Nombre de
usuario

Nombre del
proyecto

Credencial
del proyecto

Ilustración 4.15 - Modificación del encabezado ThingESP32

Las librerías `#include <Wifi.h>` e `#include <ThingESP.h>` no sufrirán ningún cambio, pero en la parte del código `ThingESP32 thing()` los datos variarán dependiendo de cómo se hayan registrado. En el caso del nombre de usuario se debe usar el nombre con el cual nos registramos en ThingESP, en el apartado del nombre y credencial del proyecto escribiremos los nombres que se hayan escogido para el nombre del proyecto anteriormente creado.

Otro punto para recalcar es la última modificación que tenemos que hacerle a nuestro código. La cual consiste en la configuración de la red WiFi que estemos usando en este momento, esta configuración se debe realizar como se muestra en la *Ilustración 4.16*.

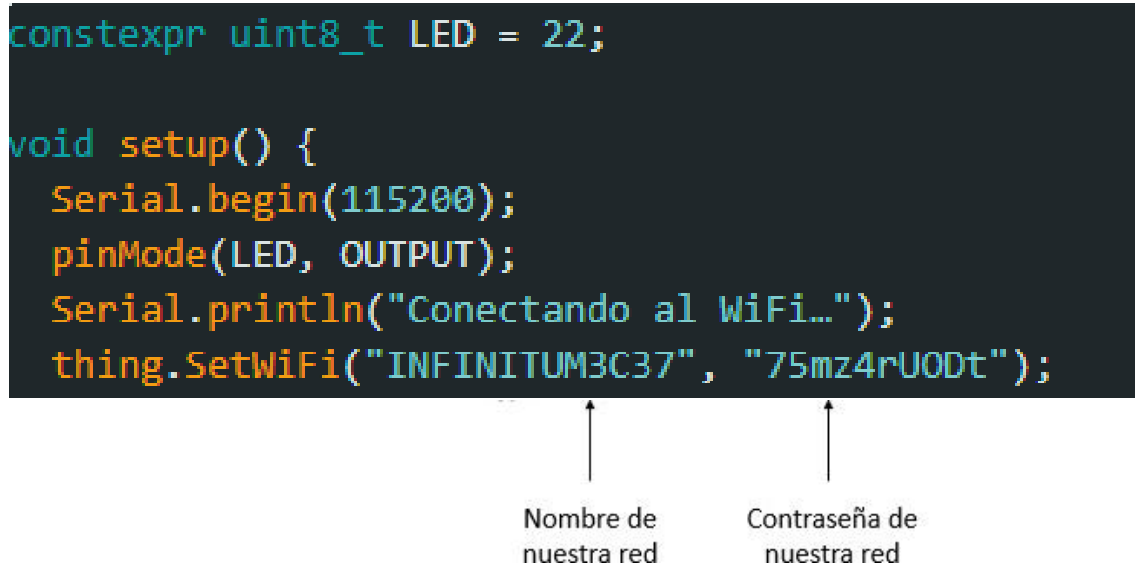


Ilustración 4.16 - Configuración de nuestra red

El siguiente paso es registrarse en la plataforma Twilio, para esto se ingresa al siguiente enlace:

Una vez ingresado a la página se dirige al apartado “Comenzar gratis”, *Ilustración 4.17*.



Ilustración 4.17 - Plataforma Twilio

Al ingresar a este apartado, lo siguiente es llenar los datos que se piden para así poder crear la cuenta, *Ilustraciones 4.18 y 4.19*.

Nota: Es importante que la contraseña cumpla con los requisitos para que funcione.

Sign up for free

Start your free trial and unlock all Twilio products, including Twilio SendGrid and Twilio Segment.

First name

Last name

Email address*

Password*

☐ Verify you are human

☐ By clicking Continue, you agree to the [Twilio Terms of Service](#) and the [Twilio Privacy Notice](#). If you are in the EEA or UK, you have read and agree to the [Electronic Communications Code Disclosures](#), if applicable.

Continue

Ilustración 4.18: Registro en Twilio

First name
Angel Alberto

Last name
Izquierdo Rodríguez

Email address*
aair020524@gmail.com

Password*
.....

Success!

Your password must contain:

- ✓ At least 8 characters
- ✓ At least 3 of the following:
 - ✓ Lower case letters (a-z)
 - ✓ Upper case letters (A-Z)
 - ✓ Numbers (0-9)
 - ✓ Special characters (e.g. !@#\$\$%^&*)
- ✓ No more than 2 identical characters in a row

Ilustración 4.19: Registro en Twilio

Una vez registrados, se envía un enlace de confirmación al correo, *Ilustración 4.20*.

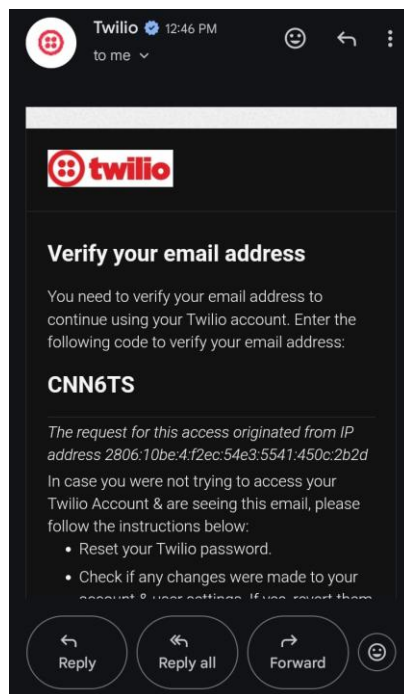


Ilustración 4.20 - Verificación de correo

Al confirmar el correo, se procede a agregar un número celular para seguidamente hacer click en “Verify”, *Ilustración 4.21*.

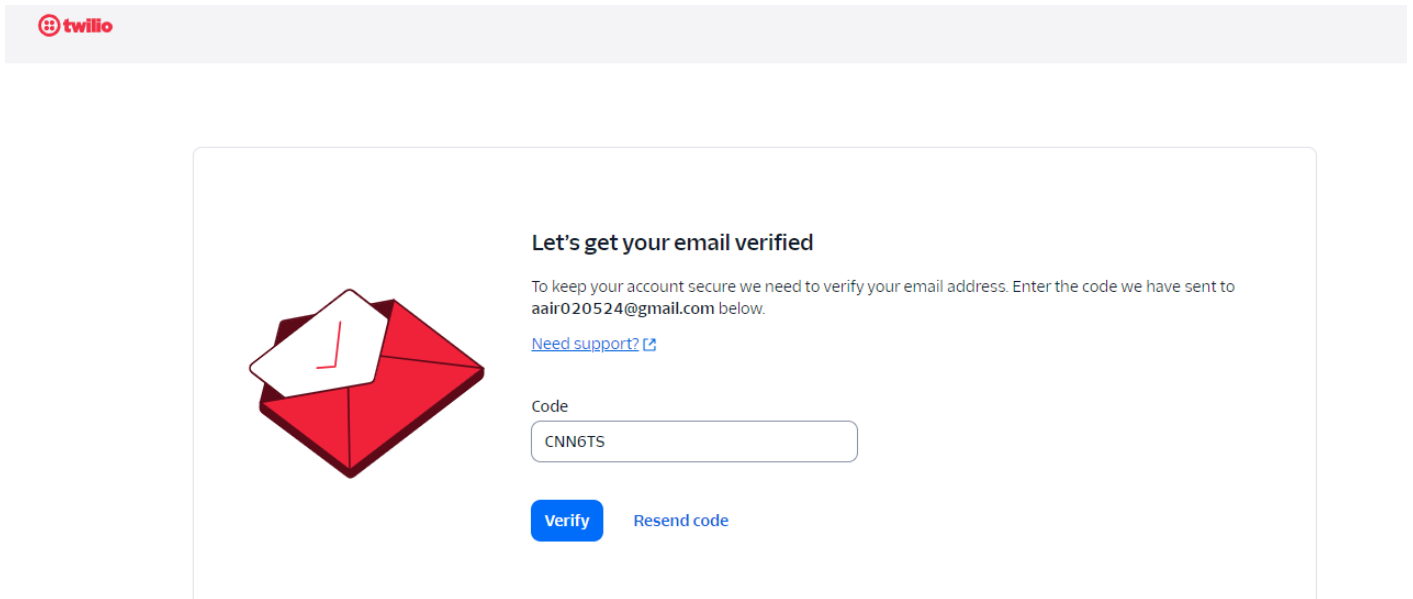


Ilustración 4.21 - Ingreso de nuestro número celular

Se recibe un código al celular y se digita a continuación para dar por finalizado el registro, *Ilustración 4.22*.

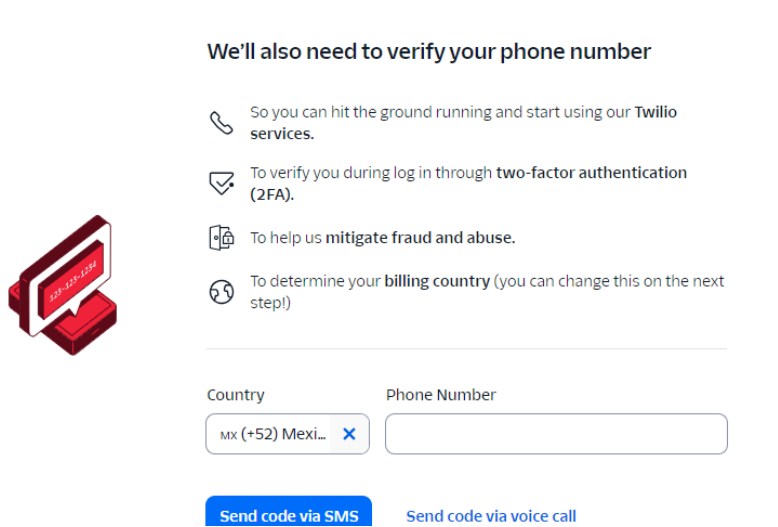


Ilustración 4.22 - Verificación de código SMS

Después de verificar el código, aparecerá la ventana de la *Ilustración 4.23*.

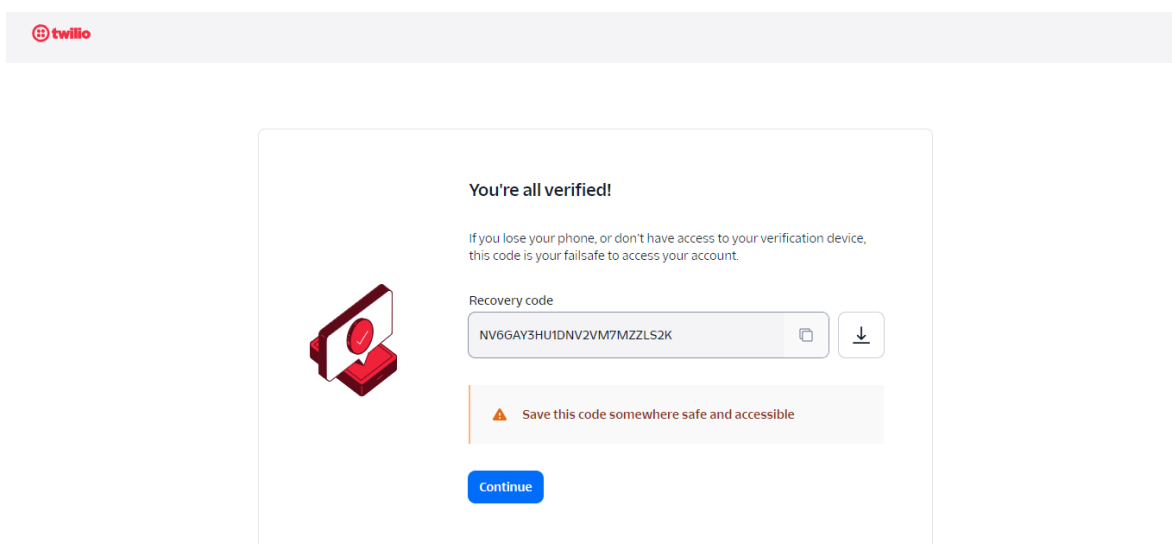


Ilustración 4.23 - Pantalla de bienvenida

Debemos bajar más en esta página de bienvenida hasta encontrar el siguiente formulario, este se responde completamente, *Ilustración 4.24*. Seguidamente presionaremos en el botón “Empezar con Twilio”.

The image shows a Twilio onboarding form. At the top, it says "Personalize your onboarding" with a star icon and a progress bar showing "66%". The main heading is "How do you want to build with Twilio?". There are four radio button options: "With code" (Customize extensively for precise needs, maximizing flexibility), "With minimal code" (Balance speed with flexibility by building on top of our code samples), "With no code at all" (Opt for simplicity with out-of-the-box solutions, requiring no code), and "With 3rd party integration" (Integrate Twilio with 3rd party tools). The "With no code at all" option is selected. At the bottom are "Back" and "Next" buttons.

Ilustración 4.24 - Formulario en Twilio

Al terminar el formulario hacer click en el apartado “Messaging”, *Ilustración 4.25*.

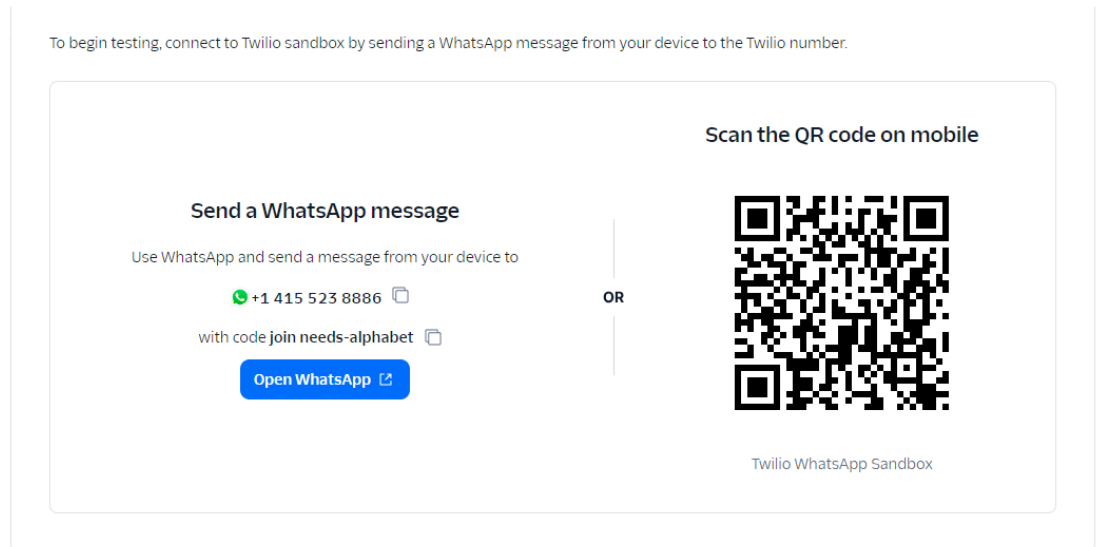
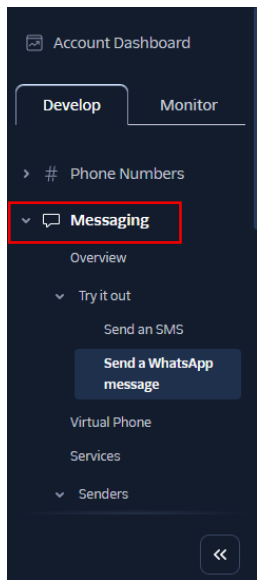


Ilustración 4.25 - Página principal de Twilio

Después de desplegar el siguiente menú, hacer click en el apartado “Try it out” y posteriormente elegir la opción “Send a Whatsapp message”, *Ilustración 4.26*.

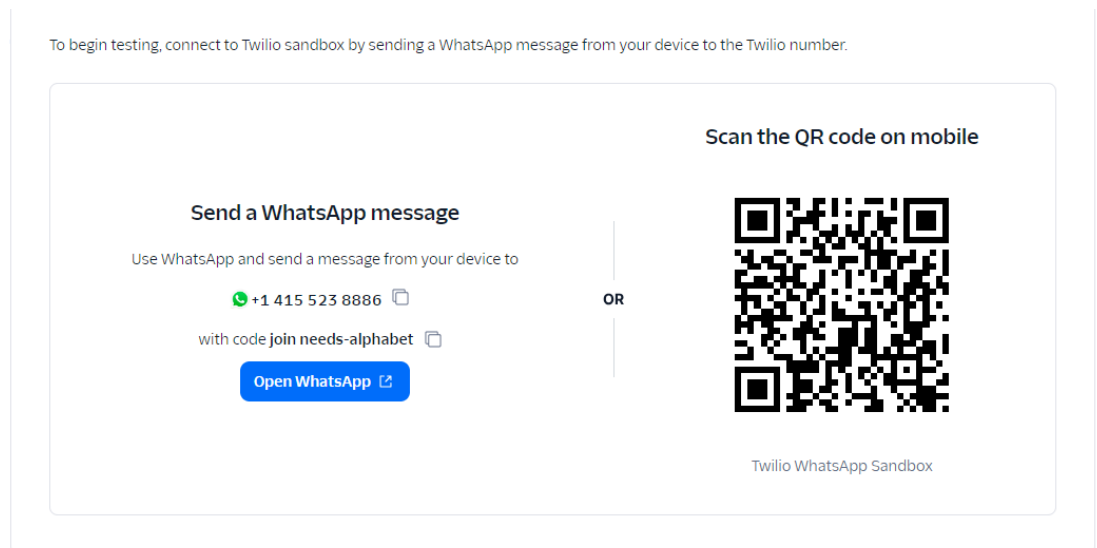
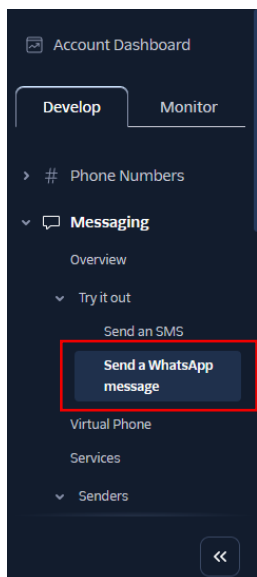


Ilustración 4.26 - Navegar hacia "Send Whatsapp message"

Al seleccionar esta opción, se nos abrirá la siguiente ventana, inmediatamente procederemos a dar click en el apartado “Sandbox Setting”, *Ilustración 4.27*.

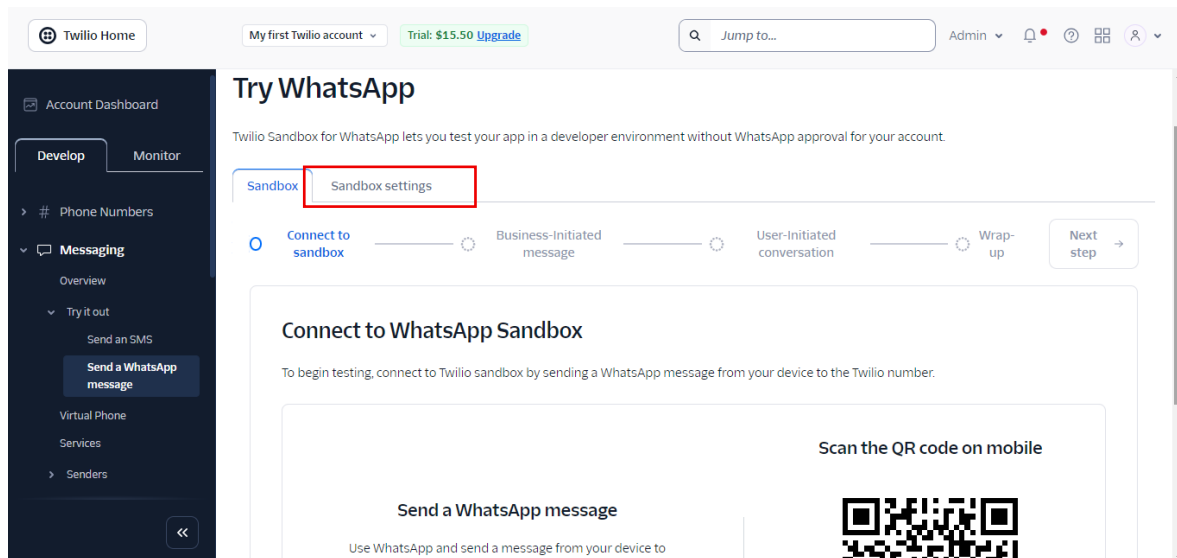


Ilustración 4.27 - Apartado Sandbox settings

En estas configuraciones, tendremos que poner el link que proporciono la Plataforma ThingESP. Este link es el anteriormente visto en la *Ilustracion 14*. Ahora que tenemos el link se copia en el apartado “When a message comes in”. Seguidamente presionaremos el botón “Save”, *Ilustración 4.28*.

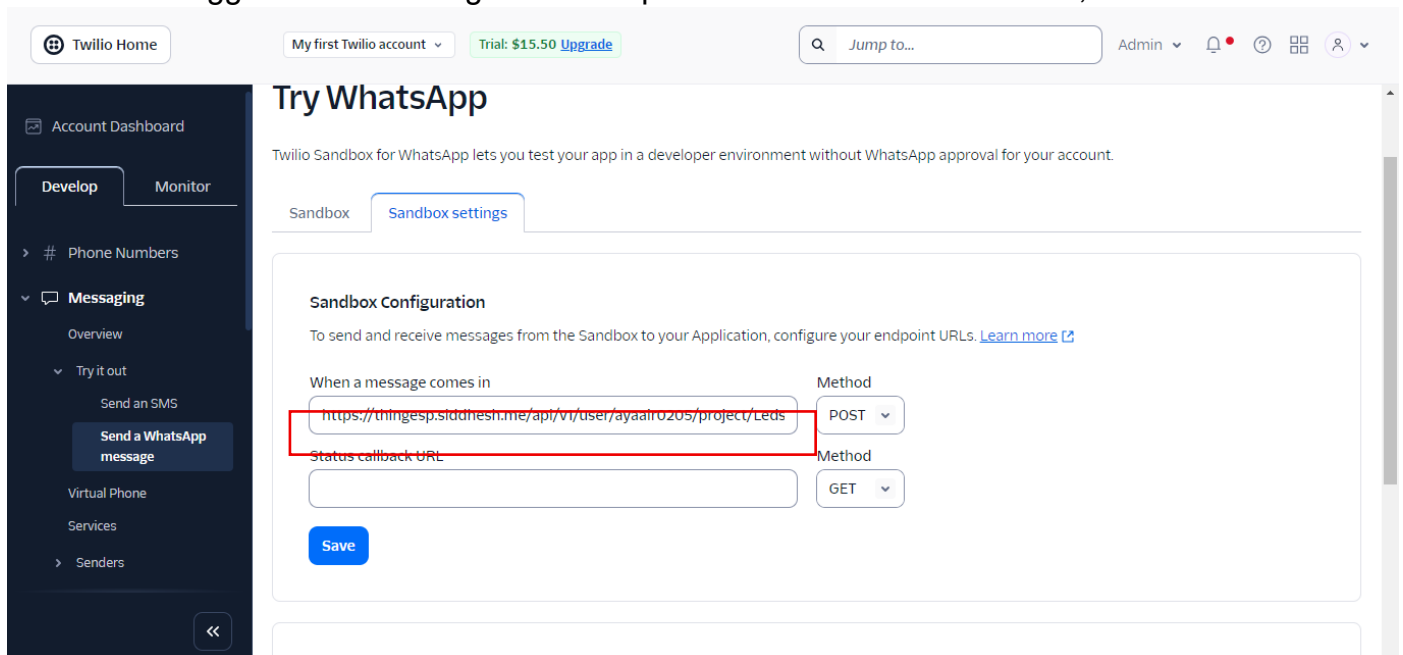


Ilustración 4.28 - configuración del apartado "When a message comes in"

Una vez guardado el código, diríjase al final de la página para buscar el número que se debe agregar a los contactos de Whatsapp y también el mensaje que debe ser enviado a este número, *Ilustración 4.29*.

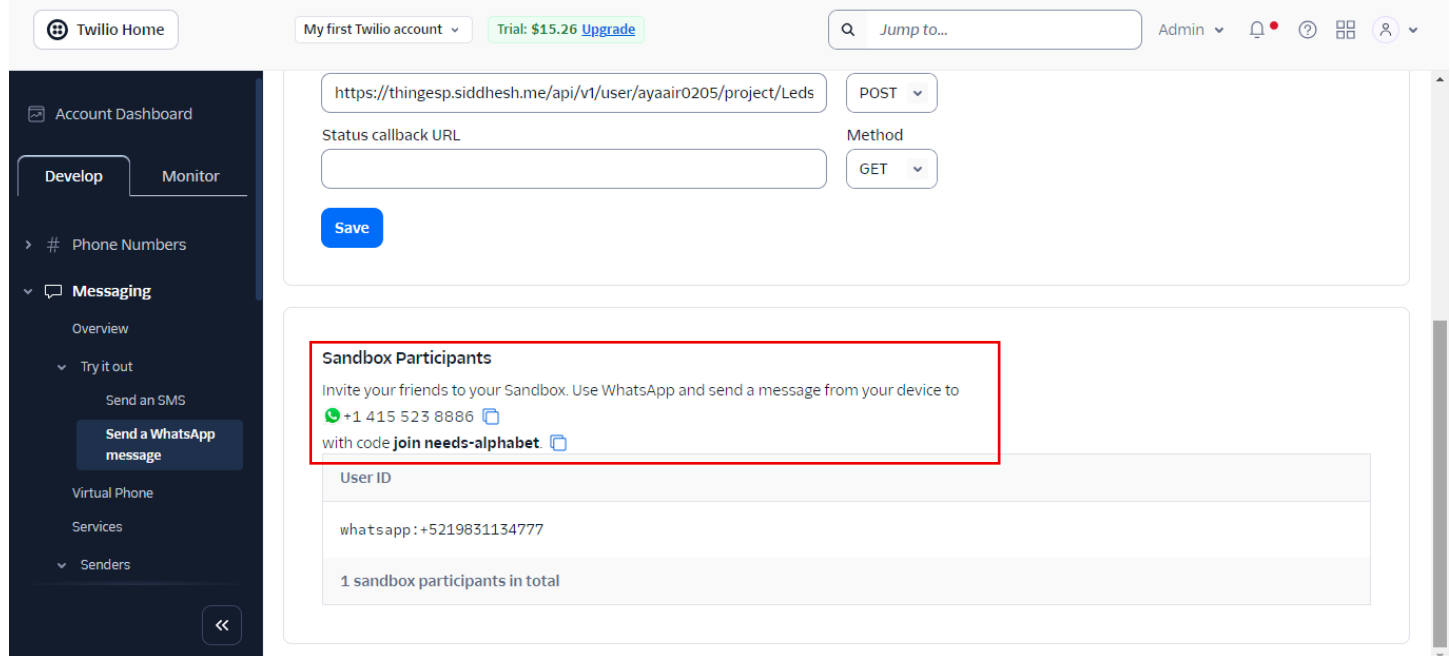


Ilustración 4.29 - Numero al cual le debemos enviar el mensaje y el mensaje que debe ser enviado

Al enviar el código, el bot nos deberá responder, si esto ocurre, quiere decir que hicimos toda la configuración de manera exitosa, *Ilustración 4.30*.

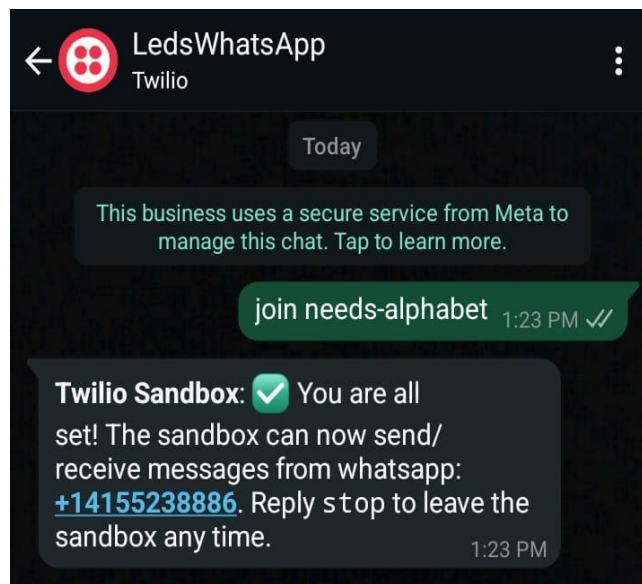


Ilustración 4.30 - Mensaje de recibido

Para finalizar, se carga el código anteriormente visto en el punto 3.2 a la tarjeta ESP32, recuerde escoger el puerto correcto y escoger la tarjeta “ESP-32 Dev Module”. Es importante haber realizado los cambios relevantes anteriormente mencionados, como lo son realizar el cambio en el nombre y contraseña de red, al igual que los cambios en el usuario: nombre del proyecto y credenciales del proyecto. De lo contrario, el código no funcionará, para cualquier duda, recuerda que estos cambios se realizaron en pasos anteriores.

```
#include <WiFi.h>
#include <ThingESP.h>

ThingESP32 thing("ayaair0205", "LedsWhatsApp", "ledonoff");
constexpr uint8_t LED = 22;

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
  Serial.println("Conectando al WiFi...");
  thing.SetWiFi("INFINITUM3C37", "75mz4rUODt");
  thing.initDevice();          // Intenta registrar el dispositivo
  Serial.println("Esperando comandos de WhatsApp");
}

String HandleResponse(String query) {
  query.toLowerCase();        // Acepta "Led ON", "LED on", etc.
  if (query == "led on") { digitalWrite(LED, HIGH); return "Done: LED Turned ON"; }
  if (query == "led off") { digitalWrite(LED, LOW); return "Done: LED Turned OFF"; }
  if (query == "led status")
    return digitalRead(LED) ? "LED is ON" : "LED is OFF";
  return "Your query was invalid.";
}

void loop() { thing.Handle(); }
```

Por último, se envían los mensajes que se capturaron en la programación de Arduino desde la plataforma de WhatsApp. Si se observa que se realiza la acción que solicitamos se puede concluir que el funcionamiento del código es exitoso, *Ilustración 4.31*.

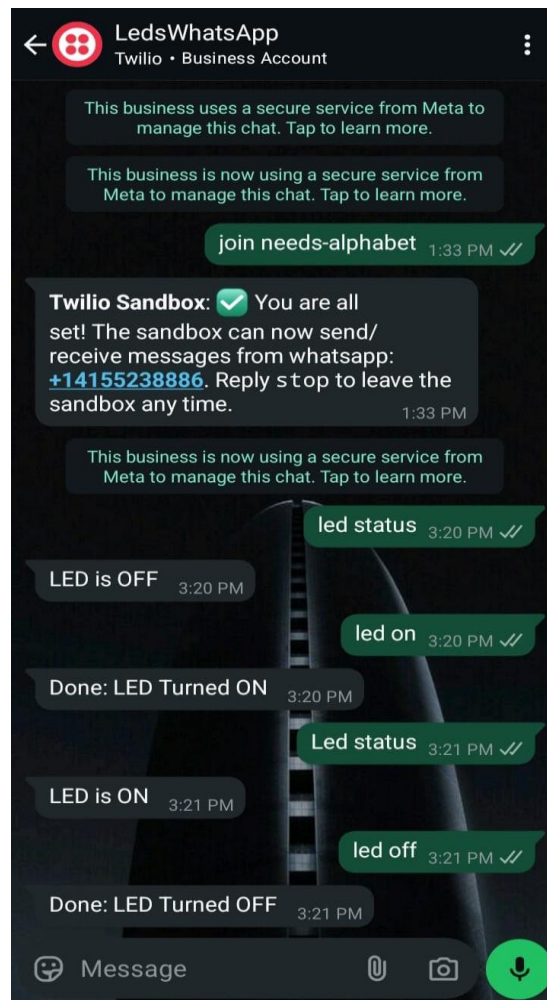


Ilustración 4.31 - Funcionamiento exitoso de nuestro código

4.2.1 Implementa el circuito del diagrama que se muestra en la *Ilustración 4.32*.



```
#include <WiFi.h>

#include <ThingESP.h>

#include <OneWire.h>

#include <DallasTemperature.h>

//

// Credenciales ThingESP & Wi-Fi

ThingESP32 thing("ayaair0205", "LedsWhatsApp", "ledonoff");

constexpr uint8_t LED_PIN = 22; // LED a controlar

constexpr uint8_t ONE_WIRE_BUS = 23; // Pin donde conectas el DS18B20

// Objetos para el sensor de temperatura
```

```

OneWire oneWire(ONE_WIRE_BUS);

DallasTemperature sensors(&oneWire);


//

void setup() {

  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);


  Serial.println("Conectando al WiFi...");
  thing.SetWiFi("INFINITUM3C37", "75mz4rUODt");


  sensors.begin();          // Inicializa DS18B20
  thing.initDevice();        // Registra el dispositivo en ThingESP
  Serial.println("Esperando comandos de WhatsApp");
}


// Maneja cada mensaje que recibes por WhatsApp
String HandleResponse(String query) {

  query.toLowerCase();      // Ignora mayúsculas/minúsculas


  // 1) Control tradicional del LED
  if (query == "led on")    { digitalWrite(LED_PIN, HIGH); return "Done: LED Turned ON"; }
  if (query == "led off")   { digitalWrite(LED_PIN, LOW);  return "Done: LED Turned OFF"; }
  if (query == "led status") { return digitalRead(LED_PIN) ? "LED is ON" : "LED is OFF"; }


  // 2) "Sex mode" (enciende el LED de forma... insinuante 😏)
  if (query == "sex mode") {

```

```

digitalWrite(LED_PIN, HIGH);

return " 🔥 Sex mode activated. Enjoy! 🔥 ";
}

// 3) Lectura de temperatura de la pecera
if (query == "temp" || query == "pecera" || query == "temperature") {
    sensors.requestTemperatures();      // Llama al sensor
    float tempC = sensors.getTempCByIndex(0);
    if (tempC == DEVICE_DISCONNECTED_C) // Sensor no responde
        return "Sensor no conectado 🙄";

    char buffer[40];
    snprintf(buffer, sizeof(buffer),
        "La pecera está a %.2f °C 🌈", tempC);
    return String(buffer);
}

return "Comando no reconocido 🙄";
}

void loop() {
    thing.Handle(); // Mantén la conexión y procesa los mensajes
}

```

Para esta práctica, se utilizará el mismo proyecto que se usó en la práctica anterior. Esto facilitará las configuraciones ya que al usar el mismo proyecto tendremos entrelazadas las aplicaciones ThingESP y Twilio.

Cargamos el código del punto 4.2 a nuestra tarjeta, recuerda realizar los cambios en las siguientes líneas de código con tus credenciales:

- ThingESP32 `thing("ayaair0205", "LedSWhatsApp", "ledonoff");`
- `thing.SetWiFi("INFINITUM3C37", "75mz4rUODt");`

Una vez cargado el código a la ESP32 solo basta con enviar el mensaje capturado en la programación al número anteriormente agregado.

NOTA: Es posible que tras días de inactividad nuestro número de teléfono se desvincule con Twilio. Si esto pasa, debemos enviarle el mismo mensaje que se encuentra en el apartado "Sandbox Settings" a nuestro bot de Twilio, *Ilustración 4.29*.

Una vez enviado el mensaje capturado en la programación, nuestro resultado será el mostrado en la *Ilustración 4.33*.



Ilustración 4.33 - Funcionamiento exitoso de nuestro código

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

REFERENCIAS

Referencias

Kymera Systems. (agosto de 2025). *What is MQTT, and Why Does it Matter?* Obtenido de <https://www.kymerasystems.com/what-is-mqtt-and-why-does-it-matter/>

Söderby, K. (16 de Julio de 2025). *ARDUINODOCS*. Obtenido de <https://docs.arduino.cc/arduino-cloud/guides/overview/>